COMPLETE SOURCE CODE EXPLANATION

Domain Models & Enumerations

Line-by-Line Explanation of OOP Entities, Encapsulation & Inheritance

Covers 22 Model classes and 6 Type-Safe Enumerations. Explains Encapsulation, Polymorphism (Payment types), Inheritance (User hierarchy), and domain business getters.

Project: Restaurant Management System

Language: Java 17 / JavaFX / JDBC

Scope: Detailed Line-by-Line Breakdown

Table of Contents

Admin.java	src\main\java\com\restaurant\model\Admin.java
CardPayment.java	src\main\java\com\restaurant\model\CardPayment.java
CashPayment.java	src\main\java\com\restaurant\model\CashPayment.java
Cashier.java	src\main\java\com\restaurant\model\Cashier.java
Customer.java	src\main\java\com\restaurant\model\Customer.java
Employee.java	src\main\java\com\restaurant\model\Employee.java
InventoryItem.java	src\main\java\com\restaurant\model\InventoryItem.java
KitchenStaff.java	src\main\java\com\restaurant\model\KitchenStaff.java
Manager.java	src\main\java\com\restaurant\model\Manager.java
MenuCategory.java	src\main\java\com\restaurant\model\MenuCategory.java
MenuItem.java	src\main\java\com\restaurant\model\MenuItem.java
Order.java	src\main\java\com\restaurant\model\Order.java
OrderItem.java	src\main\java\com\restaurant\model\OrderItem.java
Payment.java	src\main\java\com\restaurant\model\Payment.java
Purchase.java	src\main\java\com\restaurant\model\Purchase.java
PurchaseItem.java	src\main\java\com\restaurant\model\PurchaseItem.java
Reservation.java	src\main\java\com\restaurant\model\Reservation.java
RestaurantTable.java	src\main\java\com\restaurant\model\RestaurantTable.java
Supplier.java	src\main\java\com\restaurant\model\Supplier.java
UPIPayment.java	src\main\java\com\restaurant\model\UPIPayment.java
User.java	src\main\java\com\restaurant\model\User.java
Waiter.java	src\main\java\com\restaurant\model\Waiter.java
OrderStatus.java	src\main\java\com\restaurant\enums\OrderStatus.java
PaymentMethod.java	src\main\java\com\restaurant\enums\PaymentMethod.java
PaymentStatus.java	src\main\java\com\restaurant\enums\PaymentStatus.java
ReservationStatus.java	src\main\java\com\restaurant\enums\ReservationStatus.java
TableStatus.java	src\main\java\com\restaurant\enums\TableStatus.java
UserRole.java	src\main\java\com\restaurant\enums\UserRole.java

Admin.java

Path: src\main\java\com\restaurant\model\Admin.java | Total Lines: 21 | Logical Blocks: 4

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the Admin role in the Restaurant Management architecture.

Lines 1–6 Package & Library Imports

```
1  package com.restaurant.model;
2
3  import com.restaurant.enums.UserRole;
4  import java.util.Arrays;
5  import java.util.List;
6
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 6. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 7–7 Class Declaration & Heritage

```
7  public class Admin extends User {
```

Architecture Role	Defines Admin as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 7, allowing cross-package invocation.

Lines 8–17 Class Members & Implementation

```
8
9      public Admin() {
10         this.role = UserRole.ADMIN;
11     }
12
13     public Admin(int userId, String username, String fullName, String email, String phone) {
14         super(userId, username, fullName, email, phone, UserRole.ADMIN);
15     }
16
17     @Override
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 8–17 (10 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 18–21` Method: getPermissions()

```
18        public List<String> getPermissions() {
19            return List.of("ALL", "MANAGE_USERS", "MANAGE_MENU", "MANAGE_TABLES", "MANAGE_INVENTORY", "VIE
W_REPORTS", "SYSTEM_SETTINGS");
20        }
21    }
```

Component	Method: getPermissions()
Line-by-Line Breakdown	Lines 18–21 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

CardPayment.java

Path: src\main\java\com\restaurant\model\CardPayment.java | Total Lines: 73 | Logical Blocks: 9

📘 COMPONENT OVERVIEW & RESPONSIBILITY

Defines and executes the CardPayment role in the Restaurant Management architecture.

Lines 1–8 Package & Library Imports

```
1  package com.restaurant.model;
2
3  import com.restaurant.enums.PaymentMethod;
4  import com.restaurant.enums.PaymentStatus;
5  import com.restaurant.exception.PaymentFailedException;
6  import com.restaurant.util.CurrencyUtil;
7  import com.restaurant.util.DateTimeUtil;
8
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 8. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 9–9 Class Declaration & Heritage

```
9  public class CardPayment extends Payment {
```

Architecture Role	Defines CardPayment as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 9, allowing cross-package invocation.

Lines 10–24 Class Members & Implementation

```
10        private String cardNumberLastFour;
11        private String approvalCode;
12
13        public CardPayment() {
14            this.paymentMethod = PaymentMethod.CREDIT_CARD;
15        }
16
17        public CardPayment(int orderId, String invoiceNumber, double amount, PaymentMethod method, int pro
cessedBy, String cardNumberLastFour, String approvalCode) {
18            super(orderId, invoiceNumber, amount, method, processedBy);
19            this.cardNumberLastFour = cardNumberLastFour;
20            this.approvalCode = approvalCode;
21            this.transactionReference = "CARD-AUTH-" + approvalCode;
22        }
23
24        @Override
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 10–24 (15 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 25–38 Method: processPayment()

```
25        public boolean processPayment() throws PaymentFailedException {
26            if (cardNumberLastFour == null || cardNumberLastFour.length() != 4) {
27                this.status = PaymentStatus.FAILED;
28                throw new PaymentFailedException("Invalid card digits. Last 4 digits required.");
29            }
30            if (approvalCode == null || approvalCode.trim().isEmpty()) {
31                this.status = PaymentStatus.FAILED;
32                throw new PaymentFailedException("Card authorization approval code missing from POS termin
al.");
33            }
34            this.status = PaymentStatus.PAID;
35            return true;
36        }
37
38        @Override
```

Component	Method: processPayment()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 25–38 (14 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 39–57 Method: generateReceipt()

```
39        public String generateReceipt(Order order) {
40            StringBuilder sb = new StringBuilder();
41            sb.append("========================================\n");
42            sb.append("         GRAND FEAST RESTAURANT          \n");
43            sb.append("        104 Gourmet Blvd, Silicon City   \n");
44            sb.append("========================================\n");
45            sb.append(String.format("Invoice No : %s\n", invoiceNumber));
46            sb.append(String.format("Order No   : %s\n", order != null ? order.getOrderNumber() : "ORD-" +
orderId));
47            sb.append(String.format("Date/Time  : %s\n", DateTimeUtil.formatDateTime(paymentDateTime)));
48            sb.append(String.format("Pay Mode   : %s (**** %s)\n", paymentMethod.getDisplayName(), cardNum
berLastFour));
49            sb.append(String.format("Auth Code  : %s\n", approvalCode));
50            sb.append("----------------------------------------\n");
51            sb.append(String.format("Total Paid : %s [APPROVED]\n", CurrencyUtil.format(amount)));
52            sb.append("----------------------------------------\n");
53            sb.append("       THANK YOU FOR DINING WITH US!     \n");
54            sb.append("========================================\n");
55            return sb.toString();
56        }
57
```

Component	Method: generateReceipt()
Line-by-Line Breakdown	Lines 39–57 (19 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 58–61 Method: getCardNumberLastFour()

```
58        public String getCardNumberLastFour() {
59            return cardNumberLastFour;
60        }
61
```

Component	Method: getCardNumberLastFour()
Line-by-Line Breakdown	Lines 58–61 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 62–65 Method: setCardNumberLastFour()

```
62        public void setCardNumberLastFour(String cardNumberLastFour) {
63            this.cardNumberLastFour = cardNumberLastFour;
64        }
65
```

Component	Method: setCardNumberLastFour()
Line-by-Line Breakdown	Lines 62–65 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 66–69 Method: getApprovalCode()

```
66        public String getApprovalCode() {
67            return approvalCode;
68        }
69
```

Component	Method: getApprovalCode()
Line-by-Line Breakdown	Lines 66–69 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 70–73 Method: setApprovalCode()

```
70        public void setApprovalCode(String approvalCode) {
71            this.approvalCode = approvalCode;
72        }
73    }
```

Component	Method: setApprovalCode()
Line-by-Line Breakdown	Lines 70–73 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

CashPayment.java

Path: src\main\java\com\restaurant\model\CashPayment.java | Total Lines: 72 | Logical Blocks: 9

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the CashPayment role in the Restaurant Management architecture.

Lines 1–8 Package & Library Imports

```
1  package com.restaurant.model;
2
3  import com.restaurant.enums.PaymentMethod;
4  import com.restaurant.enums.PaymentStatus;
5  import com.restaurant.exception.PaymentFailedException;
6  import com.restaurant.util.CurrencyUtil;
7  import com.restaurant.util.DateTimeUtil;
8
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 8. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 9–9 Class Declaration & Heritage

```
9  public class CashPayment extends Payment {
```

Architecture Role	Defines CashPayment as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 9, allowing cross-package invocation.

Lines 10–24 Class Members & Implementation

```
10        private double cashTendered;
11        private double changeDue;
12
13        public CashPayment() {
14            this.paymentMethod = PaymentMethod.CASH;
15        }
16
17        public CashPayment(int orderId, String invoiceNumber, double amount, int processedBy, double cashT
endered) {
18            super(orderId, invoiceNumber, amount, PaymentMethod.CASH, processedBy);
19            this.cashTendered = cashTendered;
20            this.changeDue = Math.max(0, cashTendered - amount);
21            this.transactionReference = "CASH-TXN-" + System.currentTimeMillis() % 1000000;
22        }
23
24        @Override
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 10–24 (15 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 25–36 Method: processPayment()

```
25        public boolean processPayment() throws PaymentFailedException {
26            if (cashTendered < amount) {
27                this.status = PaymentStatus.FAILED;
28                throw new PaymentFailedException(String.format("Insufficient cash tendered! Required: %s,
Received: %s",
29                        CurrencyUtil.format(amount), CurrencyUtil.format(cashTendered)));
30            }
31            this.changeDue = cashTendered - amount;
32            this.status = PaymentStatus.PAID;
33            return true;
34        }
35
36        @Override
```

Component	Method: processPayment()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 25–36 (12 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 37–56 Method: generateReceipt()

```
37      public String generateReceipt(Order order) {
38          StringBuilder sb = new StringBuilder();
39          sb.append("========================================\n");
40          sb.append("         GRAND FEAST RESTAURANT          \n");
41          sb.append("        104 Gourmet Blvd, Silicon City   \n");
42          sb.append("========================================\n");
43          sb.append(String.format("Invoice No : %s\n", invoiceNumber));
44          sb.append(String.format("Order No   : %s\n", order != null ? order.getOrderNumber() : "ORD-" +
orderId));
45          sb.append(String.format("Date/Time  : %s\n", DateTimeUtil.formatDateTime(paymentDateTime)));
46          sb.append(String.format("Pay Mode   : %s\n", paymentMethod.getDisplayName()));
47          sb.append("----------------------------------------\n");
48          sb.append(String.format("Bill Amount  : %s\n", CurrencyUtil.format(amount)));
49          sb.append(String.format("Cash Given   : %s\n", CurrencyUtil.format(cashTendered)));
50          sb.append(String.format("Change Back  : %s\n", CurrencyUtil.format(changeDue)));
51          sb.append("----------------------------------------\n");
52          sb.append("       THANK YOU FOR DINING WITH US!     \n");
53          sb.append("========================================\n");
54          return sb.toString();
55      }
56
```

Component	Method: generateReceipt()
Line-by-Line Breakdown	Lines 37–56 (20 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 57–60 Method: getCashTendered()

```
57      public double getCashTendered() {
58          return cashTendered;
59      }
60
```

Component	Method: getCashTendered()
Line-by-Line Breakdown	Lines 57–60 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 61–64 Method: setCashTendered()

```
61      public void setCashTendered(double cashTendered) {
62          this.cashTendered = cashTendered;
63      }
64
```

Component	Method: setCashTendered()
Line-by-Line Breakdown	Lines 61–64 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 65–68 Method: getChangeDue()

```
65        public double getChangeDue() {
66            return changeDue;
67        }
68
```

Component	Method: getChangeDue()
Line-by-Line Breakdown	Lines 65–68 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 69–72 Method: setChangeDue()

```
69        public void setChangeDue(double changeDue) {
70            this.changeDue = changeDue;
71        }
72    }
```

Component	Method: setChangeDue()
Line-by-Line Breakdown	Lines 69–72 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Cashier.java

Path: src\main\java\com\restaurant\model\Cashier.java | Total Lines: 20 | Logical Blocks: 4

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the Cashier role in the Restaurant Management architecture.

Lines 1–5 Package & Library Imports

```
1  package com.restaurant.model;
2
3  import com.restaurant.enums.UserRole;
4  import java.util.List;
5
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 5. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 6–6 Class Declaration & Heritage

```
6  public class Cashier extends User {
```

Architecture Role	Defines Cashier as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 6, allowing cross-package invocation.

Lines 7–16 Class Members & Implementation

```
7
8      public Cashier() {
9          this.role = UserRole.CASHIER;
10     }
11
12     public Cashier(int userId, String username, String fullName, String email, String phone) {
13         super(userId, username, fullName, email, phone, UserRole.CASHIER);
14     }
15
16     @Override
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 7–16 (10 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 17–20` Method: getPermissions()

```
17        public List<String> getPermissions() {
18            return List.of("VIEW_COMPLETED_ORDERS", "GENERATE_BILLS", "APPLY_DISCOUNTS", "PROCESS_PAYMENT
S", "PRINT_RECEIPTS", "VIEW_TRANSACTIONS");
19        }
20    }
```

Component	Method: getPermissions()
Line-by-Line Breakdown	Lines 17–20 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Customer.java

Path: src\main\java\com\restaurant\model\Customer.java | Total Lines: 128 | Logical Blocks: 25

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the Customer role in the Restaurant Management architecture.

Lines 1–7 Package & Library Imports

```
1  package com.restaurant.model;
2
3  import java.time.LocalDateTime;
4
5  /**
6   * Customer domain entity with loyalty tracking.
7   */
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 7. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 8–8 Class Declaration & Heritage

```
8  public class Customer {
```

Architecture Role	Defines Customer as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 8, allowing cross-package invocation.

Lines 9–34 Class Members & Implementation

```
9       private int customerId;
10      private String name;
11      private String phone;
12      private String email;
13      private String address;
14      private int loyaltyPoints;
15      private int totalVisits;
16      private double totalSpent;
17      private LocalDateTime createdAt;
18      private LocalDateTime updatedAt;
19
20      public Customer() {
21          this.loyaltyPoints = 0;
22          this.totalVisits = 0;
23          this.totalSpent = 0.00;
24      }
25
26      public Customer(int customerId, String name, String phone, String email, String address) {
27          this();
28          this.customerId = customerId;
29          this.name = name;
30          this.phone = phone;
31          this.email = email;
32          this.address = address;
33      }
34
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 9–34 (26 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 35–43 Method: addSpending()

```
35      public void addSpending(double amount) {
36          if (amount > 0) {
37              this.totalSpent += amount;
38              this.totalVisits += 1;
39              // 1 point per 100 spent
40              this.loyaltyPoints += (int) (amount / 100);
41          }
42      }
43
```

Component	Method: addSpending()
Line-by-Line Breakdown	Lines 35–43 (9 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 44–47 Method: getCustomerId()

```
44      public int getCustomerId() {
45          return customerId;
46      }
47
```

Component	Method: getCustomerId()
Line-by-Line Breakdown	Lines 44–47 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 48–51 Method: setCustomerId()

```
48      public void setCustomerId(int customerId) {
49          this.customerId = customerId;
50      }
51
```

Component	Method: setCustomerId()
Line-by-Line Breakdown	Lines 48–51 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 52–55 Method: getName()

```
52      public String getName() {
53          return name;
54      }
55
```

Component	Method: getName()
Line-by-Line Breakdown	Lines 52–55 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 56–59 Method: setName()

```
56      public void setName(String name) {
57          this.name = name;
58      }
59
```

Component	Method: setName()
Line-by-Line Breakdown	Lines 56–59 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 60–63 Method: getPhone()

```
60      public String getPhone() {
61          return phone;
62      }
63
```

Component	Method: getPhone()
Line-by-Line Breakdown	Lines 60–63 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 64–67 Method: setPhone()

```
64      public void setPhone(String phone) {
65          this.phone = phone;
66      }
67
```

Component	Method: setPhone()
Line-by-Line Breakdown	Lines 64–67 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 68–71 Method: getEmail()

```
68      public String getEmail() {
69          return email;
70      }
71
```

Component	Method: getEmail()
Line-by-Line Breakdown	Lines 68–71 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 72–75 Method: setEmail()

```
72      public void setEmail(String email) {
73          this.email = email;
74      }
75
```

Component	Method: setEmail()
Line-by-Line Breakdown	Lines 72–75 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 76–79 Method: getAddress()

```
76      public String getAddress() {
77          return address;
78      }
79
```

Component	Method: getAddress()
Line-by-Line Breakdown	Lines 76–79 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 80–83 Method: setAddress()

```
80      public void setAddress(String address) {
81          this.address = address;
82      }
83
```

Component	Method: setAddress()
Line-by-Line Breakdown	Lines 80–83 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 84–87 Method: getLoyaltyPoints()

```
84      public int getLoyaltyPoints() {
85          return loyaltyPoints;
86      }
87
```

Component	Method: getLoyaltyPoints()
Line-by-Line Breakdown	Lines 84–87 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 88–91 Method: setLoyaltyPoints()

```
88      public void setLoyaltyPoints(int loyaltyPoints) {
89          this.loyaltyPoints = loyaltyPoints;
90      }
91
```

Component	Method: setLoyaltyPoints()
Line-by-Line Breakdown	Lines 88–91 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 92–95 **Method: getTotalVisits()**

```
92      public int getTotalVisits() {
93          return totalVisits;
94      }
95
```

Component	Method: getTotalVisits()
Line-by-Line Breakdown	Lines 92–95 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 96–99 **Method: setTotalVisits()**

```
96      public void setTotalVisits(int totalVisits) {
97          this.totalVisits = totalVisits;
98      }
99
```

Component	Method: setTotalVisits()
Line-by-Line Breakdown	Lines 96–99 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 100–103 **Method: getTotalSpent()**

```
100     public double getTotalSpent() {
101         return totalSpent;
102     }
103
```

Component	Method: getTotalSpent()
Line-by-Line Breakdown	Lines 100–103 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 104–107 **Method: setTotalSpent()**

```
104     public void setTotalSpent(double totalSpent) {
105         this.totalSpent = totalSpent;
106     }
107
```

Component	Method: setTotalSpent()
Line-by-Line Breakdown	Lines 104–107 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 108–111` Method: getCreatedAt()

```
108     public LocalDateTime getCreatedAt() {
109         return createdAt;
110     }
111
```

Component	Method: getCreatedAt()
Line-by-Line Breakdown	Lines 108–111 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 112–115` Method: setCreatedAt()

```
112     public void setCreatedAt(LocalDateTime createdAt) {
113         this.createdAt = createdAt;
114     }
115
```

Component	Method: setCreatedAt()
Line-by-Line Breakdown	Lines 112–115 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 116–119` Method: getUpdatedAt()

```
116     public LocalDateTime getUpdatedAt() {
117         return updatedAt;
118     }
119
```

Component	Method: getUpdatedAt()
Line-by-Line Breakdown	Lines 116–119 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 120–124` Method: setUpdatedAt()

```
120     public void setUpdatedAt(LocalDateTime updatedAt) {
121         this.updatedAt = updatedAt;
122     }
123
124     @Override
```

Component	Method: setUpdatedAt()
Line-by-Line Breakdown	Lines 120–124 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 125–128` Method: toString()

```
125       public String toString() {
126           return name + " (" + phone + ")";
127       }
128   }
```

Component	Method: toString()
Line-by-Line Breakdown	Lines 125–128 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Employee.java

Path: src\main\java\com\restaurant\model\Employee.java | Total Lines: 88 | Logical Blocks: 19

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the Employee role in the Restaurant Management architecture.

Lines 1–4 Package & Library Imports

```
1  package com.restaurant.model;
2
3  import java.time.LocalDate;
4
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 4. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 5–5 Class Declaration & Heritage

```
5  public class Employee {
```

Architecture Role	Defines Employee as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 5, allowing cross-package invocation.

Lines 6–24 Class Members & Implementation

```
6       private int employeeId;
7       private int userId;
8       private String designation;
9       private double salary;
10      private LocalDate hireDate;
11      private String shiftTiming;
12      private String emergencyContact;
13      private User user;
14
15      public Employee() {}
16
17      public Employee(int employeeId, int userId, String designation, double salary, LocalDate hireDate)
{
18          this.employeeId = employeeId;
19          this.userId = userId;
20          this.designation = designation;
21          this.salary = salary;
22          this.hireDate = hireDate;
23      }
24
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 6–24 (19 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 25–28 Method: getEmployeeId()

```
25      public int getEmployeeId() {
26          return employeeId;
27      }
28
```

Component	Method: getEmployeeId()
Line-by-Line Breakdown	Lines 25–28 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 29–32 Method: setEmployeeId()

```
29      public void setEmployeeId(int employeeId) {
30          this.employeeId = employeeId;
31      }
32
```

Component	Method: setEmployeeId()
Line-by-Line Breakdown	Lines 29–32 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 33–36 Method: getUserId()

```
33      public int getUserId() {
34          return userId;
35      }
36
```

Component	Method: getUserId()
Line-by-Line Breakdown	Lines 33–36 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 37–40 Method: setUserId()

```
37      public void setUserId(int userId) {
38          this.userId = userId;
39      }
40
```

Component	Method: setUserId()
Line-by-Line Breakdown	Lines 37–40 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 41–44 Method: getDesignation()

```
41      public String getDesignation() {
42          return designation;
43      }
44
```

Component	Method: getDesignation()
Line-by-Line Breakdown	Lines 41–44 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 45–48 Method: setDesignation()

```
45      public void setDesignation(String designation) {
46          this.designation = designation;
47      }
48
```

Component	Method: setDesignation()
Line-by-Line Breakdown	Lines 45–48 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 49–52 Method: getSalary()

```
49      public double getSalary() {
50          return salary;
51      }
52
```

Component	Method: getSalary()
Line-by-Line Breakdown	Lines 49–52 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 53–56 Method: setSalary()

```
53      public void setSalary(double salary) {
54          this.salary = salary;
55      }
56
```

Component	Method: setSalary()
Line-by-Line Breakdown	Lines 53–56 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 57–60 Method: getHireDate()

```
57      public LocalDate getHireDate() {
58          return hireDate;
59      }
60
```

Component	Method: getHireDate()
Line-by-Line Breakdown	Lines 57–60 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 61–64 Method: setHireDate()

```
61      public void setHireDate(LocalDate hireDate) {
62          this.hireDate = hireDate;
63      }
64
```

Component	Method: setHireDate()
Line-by-Line Breakdown	Lines 61–64 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 65–68 Method: getShiftTiming()

```
65      public String getShiftTiming() {
66          return shiftTiming;
67      }
68
```

Component	Method: getShiftTiming()
Line-by-Line Breakdown	Lines 65–68 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 69–72 Method: setShiftTiming()

```
69      public void setShiftTiming(String shiftTiming) {
70          this.shiftTiming = shiftTiming;
71      }
72
```

Component	Method: setShiftTiming()
Line-by-Line Breakdown	Lines 69–72 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 73–76 Method: getEmergencyContact()

```
73      public String getEmergencyContact() {
74          return emergencyContact;
75      }
76
```

Component	Method: getEmergencyContact()
Line-by-Line Breakdown	Lines 73–76 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 77–80 Method: setEmergencyContact()

```
77      public void setEmergencyContact(String emergencyContact) {
78          this.emergencyContact = emergencyContact;
79      }
80
```

Component	Method: setEmergencyContact()
Line-by-Line Breakdown	Lines 77–80 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 81–84` Method: getUser()

```
81        public User getUser() {
82            return user;
83        }
84
```

Component	Method: getUser()
Line-by-Line Breakdown	Lines 81–84 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 85–88` Method: setUser()

```
85        public void setUser(User user) {
86            this.user = user;
87        }
88    }
```

Component	Method: setUser()
Line-by-Line Breakdown	Lines 85–88 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

InventoryItem.java

Path: src\main\java\com\restaurant\model\InventoryItem.java | Total Lines: 131 | Logical Blocks: 27

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the InventoryItem role in the Restaurant Management architecture.

Lines 1–4 Package & Library Imports

```
1  package com.restaurant.model;
2
3  import java.time.LocalDateTime;
4
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 4. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 5–5 Class Declaration & Heritage

```
5  public class InventoryItem {
```

Architecture Role	Defines InventoryItem as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 5, allowing cross-package invocation.

Lines 6–34 Class Members & Implementation

```
6       private int inventoryId;
7       private String ingredientName;
8       private String category;
9       private double quantity;
10      private String unit;
11      private double minimumStock;
12      private double unitCost;
13      private Integer supplierId;
14      private String supplierName;
15      private LocalDateTime lastUpdated;
16
17      public InventoryItem() {
18          this.quantity = 0;
19          this.minimumStock = 5.0;
20          this.unit = "kg";
21      }
22
23      public InventoryItem(int inventoryId, String ingredientName, String category, double quantity, Str
ing unit, double minimumStock, double unitCost, Integer supplierId) {
24          this();
25          this.inventoryId = inventoryId;
26          this.ingredientName = ingredientName;
27          this.category = category;
28          this.quantity = quantity;
29          this.unit = unit;
30          this.minimumStock = minimumStock;
31          this.unitCost = unitCost;
32          this.supplierId = supplierId;
33      }
34
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 6–34 (29 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 35–38 Method: isLowStock()

```
35      public boolean isLowStock() {
36          return this.quantity <= this.minimumStock;
37      }
38
```

Component	Method: isLowStock()
Line-by-Line Breakdown	Lines 35–38 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 39–42 Method: deductStock()

```
39      public void deductStock(double amount) {
40          this.quantity = Math.max(0, this.quantity - amount);
41      }
42
```

Component	Method: deductStock()
Line-by-Line Breakdown	Lines 39–42 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 43–46 Method: addStock()

```
43      public void addStock(double amount) {
44          this.quantity += amount;
45      }
46
```

Component	Method: addStock()
Line-by-Line Breakdown	Lines 43–46 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 47–50 Method: getInventoryId()

```
47      public int getInventoryId() {
48          return inventoryId;
49      }
50
```

Component	Method: getInventoryId()
Line-by-Line Breakdown	Lines 47–50 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 51–54 Method: setInventoryId()

```
51      public void setInventoryId(int inventoryId) {
52          this.inventoryId = inventoryId;
53      }
54
```

Component	Method: setInventoryId()
Line-by-Line Breakdown	Lines 51–54 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 55–58 Method: getIngredientName()

```
55      public String getIngredientName() {
56          return ingredientName;
57      }
58
```

Component	Method: getIngredientName()
Line-by-Line Breakdown	Lines 55–58 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 59–62 Method: setIngredientName()

```
59      public void setIngredientName(String ingredientName) {
60          this.ingredientName = ingredientName;
61      }
62
```

Component	Method: setIngredientName()
Line-by-Line Breakdown	Lines 59–62 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 63–66 Method: getCategory()

```
63      public String getCategory() {
64          return category;
65      }
66
```

Component	Method: getCategory()
Line-by-Line Breakdown	Lines 63–66 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 67–70 Method: setCategory()

```
67      public void setCategory(String category) {
68          this.category = category;
69      }
70
```

Component	Method: setCategory()
Line-by-Line Breakdown	Lines 67–70 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 71–74 Method: getQuantity()

```
71      public double getQuantity() {
72          return quantity;
73      }
74
```

Component	Method: getQuantity()
Line-by-Line Breakdown	Lines 71–74 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 75–78 Method: setQuantity()

```
75      public void setQuantity(double quantity) {
76          this.quantity = quantity;
77      }
78
```

Component	Method: setQuantity()
Line-by-Line Breakdown	Lines 75–78 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 79–82 Method: getUnit()

```
79      public String getUnit() {
80          return unit;
81      }
82
```

Component	Method: getUnit()
Line-by-Line Breakdown	Lines 79–82 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 83–86 Method: setUnit()

```
83      public void setUnit(String unit) {
84          this.unit = unit;
85      }
86
```

Component	Method: setUnit()
Line-by-Line Breakdown	Lines 83–86 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 87–90 Method: getMinimumStock()

```
87      public double getMinimumStock() {
88          return minimumStock;
89      }
90
```

Component	Method: getMinimumStock()
Line-by-Line Breakdown	Lines 87–90 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 91–94 Method: setMinimumStock()

```
91      public void setMinimumStock(double minimumStock) {
92          this.minimumStock = minimumStock;
93      }
94
```

Component	Method: setMinimumStock()
Line-by-Line Breakdown	Lines 91–94 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 95–98 Method: getUnitCost()

```
95      public double getUnitCost() {
96          return unitCost;
97      }
98
```

Component	Method: getUnitCost()
Line-by-Line Breakdown	Lines 95–98 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 99–102 Method: setUnitCost()

```
99      public void setUnitCost(double unitCost) {
100         this.unitCost = unitCost;
101     }
102
```

Component	Method: setUnitCost()
Line-by-Line Breakdown	Lines 99–102 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 103–106` **Method: getSupplierId()**

```
103       public Integer getSupplierId() {
104           return supplierId;
105       }
106
```

Component	Method: getSupplierId()
Line-by-Line Breakdown	Lines 103–106 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 107–110` **Method: setSupplierId()**

```
107       public void setSupplierId(Integer supplierId) {
108           this.supplierId = supplierId;
109       }
110
```

Component	Method: setSupplierId()
Line-by-Line Breakdown	Lines 107–110 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 111–114` **Method: getSupplierName()**

```
111       public String getSupplierName() {
112           return supplierName;
113       }
114
```

Component	Method: getSupplierName()
Line-by-Line Breakdown	Lines 111–114 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 115–118` **Method: setSupplierName()**

```
115       public void setSupplierName(String supplierName) {
116           this.supplierName = supplierName;
117       }
118
```

Component	Method: setSupplierName()
Line-by-Line Breakdown	Lines 115–118 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 119–122 Method: getLastUpdated()

```
119        public LocalDateTime getLastUpdated() {
120            return lastUpdated;
121        }
122
```

Component	Method: getLastUpdated()
Line-by-Line Breakdown	Lines 119–122 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 123–127 Method: setLastUpdated()

```
123        public void setLastUpdated(LocalDateTime lastUpdated) {
124            this.lastUpdated = lastUpdated;
125        }
126
127        @Override
```

Component	Method: setLastUpdated()
Line-by-Line Breakdown	Lines 123–127 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 128–131 Method: toString()

```
128        public String toString() {
129            return ingredientName + " (" + quantity + " " + unit + ")";
130        }
131    }
```

Component	Method: toString()
Line-by-Line Breakdown	Lines 128–131 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

KitchenStaff.java

Path: src\main\java\com\restaurant\model\KitchenStaff.java | Total Lines: 20 | Logical Blocks: 4

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the KitchenStaff role in the Restaurant Management architecture.

Lines 1–5 Package & Library Imports

```
1  package com.restaurant.model;
2
3  import com.restaurant.enums.UserRole;
4  import java.util.List;
5
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 5. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 6–6 Class Declaration & Heritage

```
6  public class KitchenStaff extends User {
```

Architecture Role	Defines KitchenStaff as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 6, allowing cross-package invocation.

Lines 7–16 Class Members & Implementation

```
7
8      public KitchenStaff() {
9          this.role = UserRole.KITCHEN_STAFF;
10     }
11
12     public KitchenStaff(int userId, String username, String fullName, String email, String phone) {
13         super(userId, username, fullName, email, phone, UserRole.KITCHEN_STAFF);
14     }
15
16     @Override
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 7–16 (10 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 17–20` Method: getPermissions()

```
17        public List<String> getPermissions() {
18            return List.of("VIEW_INCOMING_ORDERS", "UPDATE_ORDER_STATUS", "VIEW_ORDER_ITEMS", "MARK_ORDER_
READY");
19        }
20    }
```

Component	Method: getPermissions()
Line-by-Line Breakdown	Lines 17–20 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Manager.java

Path: src\main\java\com\restaurant\model\Manager.java | Total Lines: 20 | Logical Blocks: 4

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the Manager role in the Restaurant Management architecture.

Lines 1–5 Package & Library Imports

```
1   package com.restaurant.model;
2
3   import com.restaurant.enums.UserRole;
4   import java.util.List;
5
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 5. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 6–6 Class Declaration & Heritage

```
6   public class Manager extends User {
```

Architecture Role	Defines Manager as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 6, allowing cross-package invocation.

Lines 7–16 Class Members & Implementation

```
7
8       public Manager() {
9           this.role = UserRole.MANAGER;
10      }
11
12      public Manager(int userId, String username, String fullName, String email, String phone) {
13          super(userId, username, fullName, email, phone, UserRole.MANAGER);
14      }
15
16      @Override
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 7–16 (10 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 17–20` Method: getPermissions()

```
17        public List<String> getPermissions() {
18            return List.of("MANAGE_MENU", "MANAGE_TABLES", "MANAGE_RESERVATIONS", "MANAGE_INVENTORY", "MAN
AGE_SUPPLIERS", "VIEW_REPORTS", "VIEW_SALES");
19        }
20    }
```

Component	Method: getPermissions()
Line-by-Line Breakdown	Lines 17–20 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

MenuCategory.java

Path: src\main\java\com\restaurant\model\MenuCategory.java | Total Lines: 54 | Logical Blocks: 12

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the MenuCategory role in the Restaurant Management architecture.

Lines 1–2 Package & Library Imports

```
1   package com.restaurant.model;
2
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 2. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 3–3 Class Declaration & Heritage

```
3   public class MenuCategory {
```

Architecture Role	Defines MenuCategory as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 3, allowing cross-package invocation.

Lines 4–17 Class Members & Implementation

```
4       private int categoryId;
5       private String name;
6       private String description;
7       private int displayOrder;
8
9       public MenuCategory() {}
10
11      public MenuCategory(int categoryId, String name, String description, int displayOrder) {
12          this.categoryId = categoryId;
13          this.name = name;
14          this.description = description;
15          this.displayOrder = displayOrder;
16      }
17
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 4–17 (14 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 18–21 Method: getCategoryId()

```
18      public int getCategoryId() {
19          return categoryId;
20      }
21
```

Component	Method: getCategoryId()
Line-by-Line Breakdown	Lines 18–21 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 22–25 Method: setCategoryId()

```
22      public void setCategoryId(int categoryId) {
23          this.categoryId = categoryId;
24      }
25
```

Component	Method: setCategoryId()
Line-by-Line Breakdown	Lines 22–25 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 26–29 Method: getName()

```
26      public String getName() {
27          return name;
28      }
29
```

Component	Method: getName()
Line-by-Line Breakdown	Lines 26–29 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 30–33 Method: setName()

```
30      public void setName(String name) {
31          this.name = name;
32      }
33
```

Component	Method: setName()
Line-by-Line Breakdown	Lines 30–33 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 34–37 Method: getDescription()

```
34      public String getDescription() {
35          return description;
36      }
37
```

Component	Method: getDescription()
Line-by-Line Breakdown	Lines 34–37 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 38–41 Method: setDescription()

```
38      public void setDescription(String description) {
39          this.description = description;
40      }
41
```

Component	Method: setDescription()
Line-by-Line Breakdown	Lines 38–41 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 42–45 Method: getDisplayOrder()

```
42      public int getDisplayOrder() {
43          return displayOrder;
44      }
45
```

Component	Method: getDisplayOrder()
Line-by-Line Breakdown	Lines 42–45 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 46–50 Method: setDisplayOrder()

```
46      public void setDisplayOrder(int displayOrder) {
47          this.displayOrder = displayOrder;
48      }
49
50      @Override
```

Component	Method: setDisplayOrder()
Line-by-Line Breakdown	Lines 46–50 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 51–54` Method: toString()

```
51        public String toString() {
52            return name;
53        }
54    }
```

Component	Method: toString()
Line-by-Line Breakdown	Lines 51–54 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

MenuItem.java

Path: src\main\java\com\restaurant\model\MenuItem.java | Total Lines: 107 | Logical Blocks: 22

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the MenuItem role in the Restaurant Management architecture.

Lines 1–4 Package & Library Imports

```
1  package com.restaurant.model;
2
3  import com.restaurant.util.CurrencyUtil;
4
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 4. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 5–5 Class Declaration & Heritage

```
5  public class MenuItem {
```

Architecture Role	Defines MenuItem as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 5, allowing cross-package invocation.

Lines 6–30 Class Members & Implementation

```
6       private int itemId;
7       private int categoryId;
8       private String categoryName;
9       private String name;
10      private String description;
11      private double price;
12      private int preparationTime; // Minutes
13      private boolean available;
14      private String imagePath;
15
16      public MenuItem() {
17          this.available = true;
18          this.preparationTime = 15;
19      }
20
21      public MenuItem(int itemId, int categoryId, String name, String description, double price, int pre
parationTime, boolean available) {
22          this.itemId = itemId;
23          this.categoryId = categoryId;
24          this.name = name;
25          this.description = description;
26          this.price = price;
27          this.preparationTime = preparationTime;
28          this.available = available;
29      }
30
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 6–30 (25 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 31–34 Method: getItemId()

```
31      public int getItemId() {
32          return itemId;
33      }
34
```

Component	Method: getItemId()
Line-by-Line Breakdown	Lines 31–34 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 35–38 Method: setItemId()

```
35      public void setItemId(int itemId) {
36          this.itemId = itemId;
37      }
38
```

Component	Method: setItemId()
Line-by-Line Breakdown	Lines 35–38 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 39–42 Method: getCategoryId()

```
39      public int getCategoryId() {
40          return categoryId;
41      }
42
```

Component	Method: getCategoryId()
Line-by-Line Breakdown	Lines 39–42 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 43–46 Method: setCategoryId()

```
43      public void setCategoryId(int categoryId) {
44          this.categoryId = categoryId;
45      }
46
```

Component	Method: setCategoryId()
Line-by-Line Breakdown	Lines 43–46 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 47–50 Method: getCategoryName()

```
47      public String getCategoryName() {
48          return categoryName;
49      }
50
```

Component	Method: getCategoryName()
Line-by-Line Breakdown	Lines 47–50 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 51–54 Method: setCategoryName()

```
51      public void setCategoryName(String categoryName) {
52          this.categoryName = categoryName;
53      }
54
```

Component	Method: setCategoryName()
Line-by-Line Breakdown	Lines 51–54 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 55–58 Method: getName()

```
55      public String getName() {
56          return name;
57      }
58
```

Component	Method: getName()
Line-by-Line Breakdown	Lines 55–58 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 59–62 Method: setName()

```
59      public void setName(String name) {
60          this.name = name;
61      }
62
```

Component	Method: setName()
Line-by-Line Breakdown	Lines 59–62 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 63–66 Method: getDescription()

```
63      public String getDescription() {
64          return description;
65      }
66
```

Component	Method: getDescription()
Line-by-Line Breakdown	Lines 63–66 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 67–70 Method: setDescription()

```
67      public void setDescription(String description) {
68          this.description = description;
69      }
70
```

Component	Method: setDescription()
Line-by-Line Breakdown	Lines 67–70 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 71–74 Method: getPrice()

```
71      public double getPrice() {
72          return price;
73      }
74
```

Component	Method: getPrice()
Line-by-Line Breakdown	Lines 71–74 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 75–78 Method: setPrice()

```
75      public void setPrice(double price) {
76          this.price = price;
77      }
78
```

Component	Method: setPrice()
Line-by-Line Breakdown	Lines 75–78 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 79–82 Method: getPreparationTime()

```
79      public int getPreparationTime() {
80          return preparationTime;
81      }
82
```

Component	Method: getPreparationTime()
Line-by-Line Breakdown	Lines 79–82 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 83–86 Method: setPreparationTime()

```
83      public void setPreparationTime(int preparationTime) {
84          this.preparationTime = preparationTime;
85      }
86
```

Component	Method: setPreparationTime()
Line-by-Line Breakdown	Lines 83–86 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 87–90 Method: isAvailable()

```
87      public boolean isAvailable() {
88          return available;
89      }
90
```

Component	Method: isAvailable()
Line-by-Line Breakdown	Lines 87–90 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 91–94 Method: setAvailable()

```
91      public void setAvailable(boolean available) {
92          this.available = available;
93      }
94
```

Component	Method: setAvailable()
Line-by-Line Breakdown	Lines 91–94 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 95–98 Method: getImagePath()

```
95      public String getImagePath() {
96          return imagePath;
97      }
98
```

Component	Method: getImagePath()
Line-by-Line Breakdown	Lines 95–98 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 99–103` Method: setImagePath()

```
 99        public void setImagePath(String imagePath) {
100            this.imagePath = imagePath;
101        }
102
103        @Override
```

Component	Method: setImagePath()
Line-by-Line Breakdown	Lines 99–103 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 104–107` Method: toString()

```
104        public String toString() {
105            return name + " (" + CurrencyUtil.format(price) + ")";
106        }
107    }
```

Component	Method: toString()
Line-by-Line Breakdown	Lines 104–107 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Order.java

Path: src\main\java\com\restaurant\model\Order.java | Total Lines: 203 | Logical Blocks: 41

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the Order role in the Restaurant Management architecture.

Lines 1–7 Package & Library Imports

```
1  package com.restaurant.model;
2
3  import com.restaurant.enums.OrderStatus;
4  import java.time.LocalDateTime;
5  import java.util.ArrayList;
6  import java.util.List;
7
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 7. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 8–8 Class Declaration & Heritage

```
8  public class Order {
```

Architecture Role	Defines Order as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 8, allowing cross-package invocation.

Lines 9–40 Class Members & Implementation

```
9       private int orderId;
10      private String orderNumber;
11      private Integer customerId;
12      private String customerName;
13      private Integer tableId;
14      private String tableNumber;
15      private int employeeId;
16      private String employeeName;
17      private LocalDateTime orderDateTime;
18      private OrderStatus status;
19      private double subtotal;
20      private double discountAmount;
21      private double taxAmount;
22      private double serviceCharge;
23      private double totalAmount;
24      private String specialNotes;
25      private List<OrderItem> items;
26
27      public Order() {
28          this.status = OrderStatus.CREATED;
29          this.orderDateTime = LocalDateTime.now();
30          this.items = new ArrayList<>();
31      }
32
33      public Order(String orderNumber, Integer customerId, Integer tableId, int employeeId) {
34          this();
35          this.orderNumber = orderNumber;
36          this.customerId = customerId;
37          this.tableId = tableId;
38          this.employeeId = employeeId;
39      }
40
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 9–40 (32 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 41–47 Method: addItem()

```
41      public void addItem(OrderItem item) {
42          if (item != null) {
43              this.items.add(item);
44              recalculateTotals();
45          }
46      }
47
```

Component	Method: addItem()
Line-by-Line Breakdown	Lines 41–47 (7 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 48–52 Method: removeItem()

```
48      public void removeItem(OrderItem item) {
49          this.items.remove(item);
50          recalculateTotals();
51      }
52
```

Component	Method: removeItem()
Line-by-Line Breakdown	Lines 48–52 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 53–60 Method: recalculateTotals()

```
53      public void recalculateTotals() {
54          this.subtotal = items.stream().mapToDouble(OrderItem::getSubtotal).sum();
55          double taxableAmount = Math.max(0, this.subtotal - this.discountAmount);
56          this.taxAmount = taxableAmount * 0.05; // 5% GST
57          this.serviceCharge = this.subtotal * 0.025; // 2.5% Service Charge
58          this.totalAmount = taxableAmount + this.taxAmount + this.serviceCharge;
59      }
60
```

Component	Method: recalculateTotals()
Line-by-Line Breakdown	Lines 53–60 (8 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 61–64 Method: getOrderId()

```
61      public int getOrderId() {
62          return orderId;
63      }
64
```

Component	Method: getOrderId()
Line-by-Line Breakdown	Lines 61–64 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 65–68 Method: setOrderId()

```
65      public void setOrderId(int orderId) {
66          this.orderId = orderId;
67      }
68
```

Component	Method: setOrderId()
Line-by-Line Breakdown	Lines 65–68 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 69–72 Method: getOrderNumber()

```
69      public String getOrderNumber() {
70          return orderNumber;
71      }
72
```

Component	Method: getOrderNumber()
Line-by-Line Breakdown	Lines 69–72 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 73–76 Method: setOrderNumber()

```
73      public void setOrderNumber(String orderNumber) {
74          this.orderNumber = orderNumber;
75      }
76
```

Component	Method: setOrderNumber()
Line-by-Line Breakdown	Lines 73–76 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 77–80 Method: getCustomerId()

```
77      public Integer getCustomerId() {
78          return customerId;
79      }
80
```

Component	Method: getCustomerId()
Line-by-Line Breakdown	Lines 77–80 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 81–84 Method: setCustomerId()

```
81      public void setCustomerId(Integer customerId) {
82          this.customerId = customerId;
83      }
84
```

Component	Method: setCustomerId()
Line-by-Line Breakdown	Lines 81–84 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 85–88 Method: getCustomerName()

```
85      public String getCustomerName() {
86          return customerName;
87      }
88
```

Component	Method: getCustomerName()
Line-by-Line Breakdown	Lines 85–88 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 89–92 Method: setCustomerName()

```
89      public void setCustomerName(String customerName) {
90          this.customerName = customerName;
91      }
92
```

Component	Method: setCustomerName()
Line-by-Line Breakdown	Lines 89–92 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 93–96 Method: getTableId()

```
93      public Integer getTableId() {
94          return tableId;
95      }
96
```

Component	Method: getTableId()
Line-by-Line Breakdown	Lines 93–96 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 97–100 Method: setTableId()

```
97      public void setTableId(Integer tableId) {
98          this.tableId = tableId;
99      }
100
```

Component	Method: setTableId()
Line-by-Line Breakdown	Lines 97–100 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 101–104 Method: getTableNumber()

```
101       public String getTableNumber() {
102           return tableNumber;
103       }
104
```

Component	Method: getTableNumber()
Line-by-Line Breakdown	Lines 101–104 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 105–108 Method: setTableNumber()

```
105       public void setTableNumber(String tableNumber) {
106           this.tableNumber = tableNumber;
107       }
108
```

Component	Method: setTableNumber()
Line-by-Line Breakdown	Lines 105–108 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 109–112 Method: getEmployeeId()

```
109       public int getEmployeeId() {
110           return employeeId;
111       }
112
```

Component	Method: getEmployeeId()
Line-by-Line Breakdown	Lines 109–112 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 113–116 Method: setEmployeeId()

```
113       public void setEmployeeId(int employeeId) {
114           this.employeeId = employeeId;
115       }
116
```

Component	Method: setEmployeeId()
Line-by-Line Breakdown	Lines 113–116 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 117–120 Method: getEmployeeName()

```
117       public String getEmployeeName() {
118           return employeeName;
119       }
120
```

Component	Method: getEmployeeName()
Line-by-Line Breakdown	Lines 117–120 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 121–124 Method: setEmployeeName()

```
121       public void setEmployeeName(String employeeName) {
122           this.employeeName = employeeName;
123       }
124
```

Component	Method: setEmployeeName()
Line-by-Line Breakdown	Lines 121–124 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 125–128 Method: getOrderDateTime()

```
125       public LocalDateTime getOrderDateTime() {
126           return orderDateTime;
127       }
128
```

Component	Method: getOrderDateTime()
Line-by-Line Breakdown	Lines 125–128 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 129–132 Method: setOrderDateTime()

```
129       public void setOrderDateTime(LocalDateTime orderDateTime) {
130           this.orderDateTime = orderDateTime;
131       }
132
```

Component	Method: setOrderDateTime()
Line-by-Line Breakdown	Lines 129–132 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 133–136` **Method: getStatus()**

```
133       public OrderStatus getStatus() {
134           return status;
135       }
136
```

Component	Method: getStatus()
Line-by-Line Breakdown	Lines 133–136 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 137–140` **Method: setStatus()**

```
137       public void setStatus(OrderStatus status) {
138           this.status = status;
139       }
140
```

Component	Method: setStatus()
Line-by-Line Breakdown	Lines 137–140 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 141–144` **Method: getSubtotal()**

```
141       public double getSubtotal() {
142           return subtotal;
143       }
144
```

Component	Method: getSubtotal()
Line-by-Line Breakdown	Lines 141–144 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 145–148` **Method: setSubtotal()**

```
145       public void setSubtotal(double subtotal) {
146           this.subtotal = subtotal;
147       }
148
```

Component	Method: setSubtotal()
Line-by-Line Breakdown	Lines 145–148 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 149–152 Method: getDiscountAmount()

```
149        public double getDiscountAmount() {
150            return discountAmount;
151        }
152
```

Component	Method: getDiscountAmount()
Line-by-Line Breakdown	Lines 149–152 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 153–157 Method: setDiscountAmount()

```
153        public void setDiscountAmount(double discountAmount) {
154            this.discountAmount = discountAmount;
155            recalculateTotals();
156        }
157
```

Component	Method: setDiscountAmount()
Line-by-Line Breakdown	Lines 153–157 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 158–161 Method: getTaxAmount()

```
158        public double getTaxAmount() {
159            return taxAmount;
160        }
161
```

Component	Method: getTaxAmount()
Line-by-Line Breakdown	Lines 158–161 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 162–165 Method: setTaxAmount()

```
162        public void setTaxAmount(double taxAmount) {
163            this.taxAmount = taxAmount;
164        }
165
```

Component	Method: setTaxAmount()
Line-by-Line Breakdown	Lines 162–165 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 166–169 Method: getServiceCharge()

```
166       public double getServiceCharge() {
167           return serviceCharge;
168       }
169
```

Component	Method: getServiceCharge()
Line-by-Line Breakdown	Lines 166–169 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 170–173 Method: setServiceCharge()

```
170       public void setServiceCharge(double serviceCharge) {
171           this.serviceCharge = serviceCharge;
172       }
173
```

Component	Method: setServiceCharge()
Line-by-Line Breakdown	Lines 170–173 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 174–177 Method: getTotalAmount()

```
174       public double getTotalAmount() {
175           return totalAmount;
176       }
177
```

Component	Method: getTotalAmount()
Line-by-Line Breakdown	Lines 174–177 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 178–181 Method: setTotalAmount()

```
178       public void setTotalAmount(double totalAmount) {
179           this.totalAmount = totalAmount;
180       }
181
```

Component	Method: setTotalAmount()
Line-by-Line Breakdown	Lines 178–181 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 182–185` Method: getSpecialNotes()

```
182       public String getSpecialNotes() {
183           return specialNotes;
184       }
185
```

Component	Method: getSpecialNotes()
Line-by-Line Breakdown	Lines 182–185 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 186–189` Method: setSpecialNotes()

```
186       public void setSpecialNotes(String specialNotes) {
187           this.specialNotes = specialNotes;
188       }
189
```

Component	Method: setSpecialNotes()
Line-by-Line Breakdown	Lines 186–189 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 190–193` Method: getItems()

```
190       public List<OrderItem> getItems() {
191           return items;
192       }
193
```

Component	Method: getItems()
Line-by-Line Breakdown	Lines 190–193 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 194–199` Method: setItems()

```
194       public void setItems(List<OrderItem> items) {
195           this.items = items != null ? items : new ArrayList<>();
196           recalculateTotals();
197       }
198
199       @Override
```

Component	Method: setItems()
Line-by-Line Breakdown	Lines 194–199 (6 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 200–203` **Method: toString()**

```
200         public String toString() {
201             return orderNumber + " [" + status + "] - Table: " + (tableNumber != null ? tableNumber : "N/
A");
202         }
203     }
```

Component	Method: toString()
Line-by-Line Breakdown	Lines 200–203 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

OrderItem.java

Path: src\main\java\com\restaurant\model\OrderItem.java | Total Lines: 112 | Logical Blocks: 23

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the OrderItem role in the Restaurant Management architecture.

Lines 1–2 Package & Library Imports

```
1  package com.restaurant.model;
2
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 2. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 3–3 Class Declaration & Heritage

```
3  public class OrderItem {
```

Architecture Role	Defines OrderItem as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 3, allowing cross-package invocation.

Lines 4–28 Class Members & Implementation

```
4       private int orderItemId;
5       private int orderId;
6       private int itemId;
7       private String itemName;
8       private int quantity;
9       private double unitPrice;
10      private String specialInstructions;
11      private double subtotal;
12      private String status;
13
14      public OrderItem() {
15          this.quantity = 1;
16          this.status = "PENDING";
17      }
18
19      public OrderItem(int itemId, String itemName, int quantity, double unitPrice, String specialInstru
ctions) {
20          this();
21          this.itemId = itemId;
22          this.itemName = itemName;
23          this.quantity = quantity;
24          this.unitPrice = unitPrice;
25          this.specialInstructions = specialInstructions;
26          this.subtotal = quantity * unitPrice;
27      }
28
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 4–28 (25 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 29–33 Method: updateQuantity()

```
29      public void updateQuantity(int newQuantity) {
30          this.quantity = newQuantity;
31          this.subtotal = this.quantity * this.unitPrice;
32      }
33
```

Component	Method: updateQuantity()
Line-by-Line Breakdown	Lines 29–33 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 34–37 Method: getOrderItemId()

```
34      public int getOrderItemId() {
35          return orderItemId;
36      }
37
```

Component	Method: getOrderItemId()
Line-by-Line Breakdown	Lines 34–37 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 38–41 Method: setOrderItemId()

```
38      public void setOrderItemId(int orderItemId) {
39          this.orderItemId = orderItemId;
40      }
41
```

Component	Method: setOrderItemId()
Line-by-Line Breakdown	Lines 38–41 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 42–45 Method: getOrderId()

```
42      public int getOrderId() {
43          return orderId;
44      }
45
```

Component	Method: getOrderId()
Line-by-Line Breakdown	Lines 42–45 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 46–49 Method: setOrderId()

```
46      public void setOrderId(int orderId) {
47          this.orderId = orderId;
48      }
49
```

Component	Method: setOrderId()
Line-by-Line Breakdown	Lines 46–49 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 50–53 Method: getItemId()

```
50      public int getItemId() {
51          return itemId;
52      }
53
```

Component	Method: getItemId()
Line-by-Line Breakdown	Lines 50–53 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 54–57 Method: setItemId()

```
54      public void setItemId(int itemId) {
55          this.itemId = itemId;
56      }
57
```

Component	Method: setItemId()
Line-by-Line Breakdown	Lines 54–57 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 58–61 Method: getItemName()

```
58      public String getItemName() {
59          return itemName;
60      }
61
```

Component	Method: getItemName()
Line-by-Line Breakdown	Lines 58–61 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 62–65 Method: setItemName()

```
62      public void setItemName(String itemName) {
63          this.itemName = itemName;
64      }
65
```

Component	Method: setItemName()
Line-by-Line Breakdown	Lines 62–65 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 66–69 Method: getQuantity()

```
66      public int getQuantity() {
67          return quantity;
68      }
69
```

Component	Method: getQuantity()
Line-by-Line Breakdown	Lines 66–69 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 70–74 Method: setQuantity()

```
70      public void setQuantity(int quantity) {
71          this.quantity = quantity;
72          this.subtotal = this.quantity * this.unitPrice;
73      }
74
```

Component	Method: setQuantity()
Line-by-Line Breakdown	Lines 70–74 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 75–78 Method: getUnitPrice()

```
75      public double getUnitPrice() {
76          return unitPrice;
77      }
78
```

Component	Method: getUnitPrice()
Line-by-Line Breakdown	Lines 75–78 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 79–83 Method: setUnitPrice()

```
79      public void setUnitPrice(double unitPrice) {
80          this.unitPrice = unitPrice;
81          this.subtotal = this.quantity * this.unitPrice;
82      }
83
```

Component	Method: setUnitPrice()
Line-by-Line Breakdown	Lines 79–83 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 84–87 Method: getSpecialInstructions()

```
84      public String getSpecialInstructions() {
85          return specialInstructions;
86      }
87
```

Component	Method: getSpecialInstructions()
Line-by-Line Breakdown	Lines 84–87 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 88–91 Method: setSpecialInstructions()

```
88      public void setSpecialInstructions(String specialInstructions) {
89          this.specialInstructions = specialInstructions;
90      }
91
```

Component	Method: setSpecialInstructions()
Line-by-Line Breakdown	Lines 88–91 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 92–95 Method: getSubtotal()

```
92      public double getSubtotal() {
93          return subtotal;
94      }
95
```

Component	Method: getSubtotal()
Line-by-Line Breakdown	Lines 92–95 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 96–99 Method: setSubtotal()

```
96      public void setSubtotal(double subtotal) {
97          this.subtotal = subtotal;
98      }
99
```

Component	Method: setSubtotal()
Line-by-Line Breakdown	Lines 96–99 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 100–103 Method: getStatus()

```
100        public String getStatus() {
101            return status;
102        }
103
```

Component	Method: getStatus()
Line-by-Line Breakdown	Lines 100–103 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 104–108 Method: setStatus()

```
104        public void setStatus(String status) {
105            this.status = status;
106        }
107
108        @Override
```

Component	Method: setStatus()
Line-by-Line Breakdown	Lines 104–108 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 109–112 Method: toString()

```
109        public String toString() {
110            return itemName + " x" + quantity + " (" + subtotal + ")";
111        }
112    }
```

Component	Method: toString()
Line-by-Line Breakdown	Lines 109–112 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Payment.java

Path: src\main\java\com\restaurant\model\Payment.java | Total Lines: 123 | Logical Blocks: 21

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the Payment role in the Restaurant Management architecture.

Lines 1–13 Package & Library Imports

```
1   package com.restaurant.model;
2
3   import com.restaurant.enums.PaymentMethod;
4   import com.restaurant.enums.PaymentStatus;
5   import com.restaurant.exception.PaymentFailedException;
6   import com.restaurant.util.CurrencyUtil;
7   import com.restaurant.util.DateTimeUtil;
8
9   import java.time.LocalDateTime;
10
11  /**
12   * Abstract base class demonstrating Polymorphism for Payment Processing.
13   */
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 13. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 14–14 Class Declaration & Heritage

```
14  public abstract class Payment {
```

Architecture Role	Defines Payment as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 14, allowing cross-package invocation.

Lines 15–51 Class Members & Implementation

```
15        protected int paymentId;
16        protected int orderId;
17        protected String invoiceNumber;
18        protected double amount;
19        protected PaymentMethod paymentMethod;
20        protected LocalDateTime paymentDateTime;
21        protected String transactionReference;
22        protected PaymentStatus status;
23        protected int processedBy;
24
25        public Payment() {
26            this.status = PaymentStatus.PENDING;
27            this.paymentDateTime = LocalDateTime.now();
28        }
29
30        public Payment(int orderId, String invoiceNumber, double amount, PaymentMethod paymentMethod, int
processedBy) {
31            this.orderId = orderId;
32            this.invoiceNumber = invoiceNumber;
33            this.amount = amount;
34            this.paymentMethod = paymentMethod;
35            this.processedBy = processedBy;
36            this.status = PaymentStatus.PENDING;
37            this.paymentDateTime = LocalDateTime.now();
38        }
39
40        /**
41         * Polymorphic payment processing method.
42         * Subclasses validate specific payment channel constraints.
43         */
44        public abstract boolean processPayment() throws PaymentFailedException;
45
46        /**
47         * Polymorphic receipt string generator tailored to payment mode.
48         */
49        public abstract String generateReceipt(Order order);
50
51        // Getters and Setters
```

Component	Class Members & Implementation
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 15–51 (37 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 52–55 Method: getPaymentId()

```
52      public int getPaymentId() {
53          return paymentId;
54      }
55
```

Component	Method: getPaymentId()
Line-by-Line Breakdown	Lines 52–55 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 56–59 Method: setPaymentId()

```
56      public void setPaymentId(int paymentId) {
57          this.paymentId = paymentId;
58      }
59
```

Component	Method: setPaymentId()
Line-by-Line Breakdown	Lines 56–59 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 60–63 Method: getOrderId()

```
60      public int getOrderId() {
61          return orderId;
62      }
63
```

Component	Method: getOrderId()
Line-by-Line Breakdown	Lines 60–63 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 64–67 Method: setOrderId()

```
64      public void setOrderId(int orderId) {
65          this.orderId = orderId;
66      }
67
```

Component	Method: setOrderId()
Line-by-Line Breakdown	Lines 64–67 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 68–71 Method: getInvoiceNumber()

```
68      public String getInvoiceNumber() {
69          return invoiceNumber;
70      }
71
```

Component	Method: getInvoiceNumber()
Line-by-Line Breakdown	Lines 68–71 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 72–75 Method: setInvoiceNumber()

```
72      public void setInvoiceNumber(String invoiceNumber) {
73          this.invoiceNumber = invoiceNumber;
74      }
75
```

Component	Method: setInvoiceNumber()
Line-by-Line Breakdown	Lines 72–75 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 76–79 Method: getAmount()

```
76      public double getAmount() {
77          return amount;
78      }
79
```

Component	Method: getAmount()
Line-by-Line Breakdown	Lines 76–79 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 80–83 Method: setAmount()

```
80      public void setAmount(double amount) {
81          this.amount = amount;
82      }
83
```

Component	Method: setAmount()
Line-by-Line Breakdown	Lines 80–83 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 84–87 Method: getPaymentMethod()

```
84      public PaymentMethod getPaymentMethod() {
85          return paymentMethod;
86      }
87
```

Component	Method: getPaymentMethod()
Line-by-Line Breakdown	Lines 84–87 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 88–91 Method: setPaymentMethod()

```
88      public void setPaymentMethod(PaymentMethod paymentMethod) {
89          this.paymentMethod = paymentMethod;
90      }
91
```

Component	Method: setPaymentMethod()
Line-by-Line Breakdown	Lines 88–91 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 92–95 Method: getPaymentDateTime()

```
92      public LocalDateTime getPaymentDateTime() {
93          return paymentDateTime;
94      }
95
```

Component	Method: getPaymentDateTime()
Line-by-Line Breakdown	Lines 92–95 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 96–99 Method: setPaymentDateTime()

```
96      public void setPaymentDateTime(LocalDateTime paymentDateTime) {
97          this.paymentDateTime = paymentDateTime;
98      }
99
```

Component	Method: setPaymentDateTime()
Line-by-Line Breakdown	Lines 96–99 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 100–103` Method: getTransactionReference()

```
100       public String getTransactionReference() {
101           return transactionReference;
102       }
103
```

Component	Method: getTransactionReference()
Line-by-Line Breakdown	Lines 100–103 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 104–107` Method: setTransactionReference()

```
104       public void setTransactionReference(String transactionReference) {
105           this.transactionReference = transactionReference;
106       }
107
```

Component	Method: setTransactionReference()
Line-by-Line Breakdown	Lines 104–107 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 108–111` Method: getStatus()

```
108       public PaymentStatus getStatus() {
109           return status;
110       }
111
```

Component	Method: getStatus()
Line-by-Line Breakdown	Lines 108–111 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 112–115` Method: setStatus()

```
112       public void setStatus(PaymentStatus status) {
113           this.status = status;
114       }
115
```

Component	Method: setStatus()
Line-by-Line Breakdown	Lines 112–115 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 116–119` Method: getProcessedBy()

```
116        public int getProcessedBy() {
117            return processedBy;
118        }
119
```

Component	Method: getProcessedBy()
Line-by-Line Breakdown	Lines 116–119 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 120–123` Method: setProcessedBy()

```
120        public void setProcessedBy(int processedBy) {
121            this.processedBy = processedBy;
122        }
123    }
```

Component	Method: setProcessedBy()
Line-by-Line Breakdown	Lines 120–123 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Purchase.java

Path: src\main\java\com\restaurant\model\Purchase.java | Total Lines: 98 | Logical Blocks: 20

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the Purchase role in the Restaurant Management architecture.

Lines 1–6 Package & Library Imports

```
1  package com.restaurant.model;
2
3  import java.time.LocalDate;
4  import java.util.ArrayList;
5  import java.util.List;
6
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 6. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 7–7 Class Declaration & Heritage

```
7  public class Purchase {
```

Architecture Role	Defines Purchase as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 7, allowing cross-package invocation.

Lines 8–29 Class Members & Implementation

```
8       private int purchaseId;
9       private int supplierId;
10      private String supplierName;
11      private LocalDate purchaseDate;
12      private double totalCost;
13      private String invoiceRef;
14      private String notes;
15      private List<PurchaseItem> items;
16
17      public Purchase() {
18          this.purchaseDate = LocalDate.now();
19          this.items = new ArrayList<>();
20      }
21
22      public Purchase(int supplierId, LocalDate purchaseDate, String invoiceRef, String notes) {
23          this();
24          this.supplierId = supplierId;
25          this.purchaseDate = purchaseDate;
26          this.invoiceRef = invoiceRef;
27          this.notes = notes;
28      }
29
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 8–29 (22 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 30–33 Method: recalculateTotal()

```
30      public void recalculateTotal() {
31          this.totalCost = items.stream().mapToDouble(PurchaseItem::getSubtotal).sum();
32      }
33
```

Component	Method: recalculateTotal()
Line-by-Line Breakdown	Lines 30–33 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 34–37 Method: getPurchaseId()

```
34      public int getPurchaseId() {
35          return purchaseId;
36      }
37
```

Component	Method: getPurchaseId()
Line-by-Line Breakdown	Lines 34–37 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 38–41 Method: setPurchaseId()

```
38      public void setPurchaseId(int purchaseId) {
39          this.purchaseId = purchaseId;
40      }
41
```

Component	Method: setPurchaseId()
Line-by-Line Breakdown	Lines 38–41 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 42–45 Method: getSupplierId()

```
42      public int getSupplierId() {
43          return supplierId;
44      }
45
```

Component	Method: getSupplierId()
Line-by-Line Breakdown	Lines 42–45 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 46–49 Method: setSupplierId()

```
46      public void setSupplierId(int supplierId) {
47          this.supplierId = supplierId;
48      }
49
```

Component	Method: setSupplierId()
Line-by-Line Breakdown	Lines 46–49 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 50–53 Method: getSupplierName()

```
50      public String getSupplierName() {
51          return supplierName;
52      }
53
```

Component	Method: getSupplierName()
Line-by-Line Breakdown	Lines 50–53 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 54–57 Method: setSupplierName()

```
54      public void setSupplierName(String supplierName) {
55          this.supplierName = supplierName;
56      }
57
```

Component	Method: setSupplierName()
Line-by-Line Breakdown	Lines 54–57 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 58–61 Method: getPurchaseDate()

```
58      public LocalDate getPurchaseDate() {
59          return purchaseDate;
60      }
61
```

Component	Method: getPurchaseDate()
Line-by-Line Breakdown	Lines 58–61 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 62–65 Method: setPurchaseDate()

```
62      public void setPurchaseDate(LocalDate purchaseDate) {
63          this.purchaseDate = purchaseDate;
64      }
65
```

Component	Method: setPurchaseDate()
Line-by-Line Breakdown	Lines 62–65 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 66–69 Method: getTotalCost()

```
66      public double getTotalCost() {
67          return totalCost;
68      }
69
```

Component	Method: getTotalCost()
Line-by-Line Breakdown	Lines 66–69 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 70–73 Method: setTotalCost()

```
70      public void setTotalCost(double totalCost) {
71          this.totalCost = totalCost;
72      }
73
```

Component	Method: setTotalCost()
Line-by-Line Breakdown	Lines 70–73 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 74–77 Method: getInvoiceRef()

```
74      public String getInvoiceRef() {
75          return invoiceRef;
76      }
77
```

Component	Method: getInvoiceRef()
Line-by-Line Breakdown	Lines 74–77 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 78–81 Method: setInvoiceRef()

```
78      public void setInvoiceRef(String invoiceRef) {
79          this.invoiceRef = invoiceRef;
80      }
81
```

Component	Method: setInvoiceRef()
Line-by-Line Breakdown	Lines 78–81 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 82–85 Method: getNotes()

```
82      public String getNotes() {
83          return notes;
84      }
85
```

Component	Method: getNotes()
Line-by-Line Breakdown	Lines 82–85 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 86–89 Method: setNotes()

```
86      public void setNotes(String notes) {
87          this.notes = notes;
88      }
89
```

Component	Method: setNotes()
Line-by-Line Breakdown	Lines 86–89 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 90–93 Method: getItems()

```
90      public List<PurchaseItem> getItems() {
91          return items;
92      }
93
```

Component	Method: getItems()
Line-by-Line Breakdown	Lines 90–93 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 94–98 Method: setItems()

```
94      public void setItems(List<PurchaseItem> items) {
95          this.items = items != null ? items : new ArrayList<>();
96          recalculateTotal();
97      }
98  }
```

Component	Method: setItems()
Line-by-Line Breakdown	Lines 94–98 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

PurchaseItem.java

Path: src\main\java\com\restaurant\model\PurchaseItem.java | Total Lines: 79 | Logical Blocks: 17

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the PurchaseItem role in the Restaurant Management architecture.

Lines 1–2 Package & Library Imports

```
1  package com.restaurant.model;
2
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 2. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 3–3 Class Declaration & Heritage

```
3  public class PurchaseItem {
```

Architecture Role	Defines PurchaseItem as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 3, allowing cross-package invocation.

Lines 4–21 Class Members & Implementation

```
4       private int purchaseItemId;
5       private int purchaseId;
6       private int inventoryId;
7       private String ingredientName;
8       private double quantity;
9       private double unitCost;
10      private double subtotal;
11
12      public PurchaseItem() {}
13
14      public PurchaseItem(int inventoryId, String ingredientName, double quantity, double unitCost) {
15          this.inventoryId = inventoryId;
16          this.ingredientName = ingredientName;
17          this.quantity = quantity;
18          this.unitCost = unitCost;
19          this.subtotal = quantity * unitCost;
20      }
21
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 4–21 (18 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 22–25 Method: getPurchaseItemId()

```
22      public int getPurchaseItemId() {
23          return purchaseItemId;
24      }
25
```

Component	Method: getPurchaseItemId()
Line-by-Line Breakdown	Lines 22–25 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 26–29 Method: setPurchaseItemId()

```
26      public void setPurchaseItemId(int purchaseItemId) {
27          this.purchaseItemId = purchaseItemId;
28      }
29
```

Component	Method: setPurchaseItemId()
Line-by-Line Breakdown	Lines 26–29 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 30–33 Method: getPurchaseId()

```
30      public int getPurchaseId() {
31          return purchaseId;
32      }
33
```

Component	Method: getPurchaseId()
Line-by-Line Breakdown	Lines 30–33 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 34–37 Method: setPurchaseId()

```
34      public void setPurchaseId(int purchaseId) {
35          this.purchaseId = purchaseId;
36      }
37
```

Component	Method: setPurchaseId()
Line-by-Line Breakdown	Lines 34–37 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 38–41 Method: getInventoryId()

```
38      public int getInventoryId() {
39          return inventoryId;
40      }
41
```

Component	Method: getInventoryId()
Line-by-Line Breakdown	Lines 38–41 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 42–45 Method: setInventoryId()

```
42      public void setInventoryId(int inventoryId) {
43          this.inventoryId = inventoryId;
44      }
45
```

Component	Method: setInventoryId()
Line-by-Line Breakdown	Lines 42–45 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 46–49 Method: getIngredientName()

```
46      public String getIngredientName() {
47          return ingredientName;
48      }
49
```

Component	Method: getIngredientName()
Line-by-Line Breakdown	Lines 46–49 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 50–53 Method: setIngredientName()

```
50      public void setIngredientName(String ingredientName) {
51          this.ingredientName = ingredientName;
52      }
53
```

Component	Method: setIngredientName()
Line-by-Line Breakdown	Lines 50–53 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 54–57 Method: getQuantity()

```
54      public double getQuantity() {
55          return quantity;
56      }
57
```

Component	Method: getQuantity()
Line-by-Line Breakdown	Lines 54–57 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 58–62 Method: setQuantity()

```
58      public void setQuantity(double quantity) {
59          this.quantity = quantity;
60          this.subtotal = quantity * this.unitCost;
61      }
62
```

Component	Method: setQuantity()
Line-by-Line Breakdown	Lines 58–62 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 63–66 Method: getUnitCost()

```
63      public double getUnitCost() {
64          return unitCost;
65      }
66
```

Component	Method: getUnitCost()
Line-by-Line Breakdown	Lines 63–66 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 67–71 Method: setUnitCost()

```
67      public void setUnitCost(double unitCost) {
68          this.unitCost = unitCost;
69          this.subtotal = this.quantity * unitCost;
70      }
71
```

Component	Method: setUnitCost()
Line-by-Line Breakdown	Lines 67–71 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 72–75 Method: getSubtotal()

```
72      public double getSubtotal() {
73          return subtotal;
74      }
75
```

Component	Method: getSubtotal()
Line-by-Line Breakdown	Lines 72–75 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 76–79 Method: setSubtotal()

```
76      public void setSubtotal(double subtotal) {
77          this.subtotal = subtotal;
78      }
79  }
```

Component	Method: setSubtotal()
Line-by-Line Breakdown	Lines 76–79 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Reservation.java

Path: src\main\java\com\restaurant\model\Reservation.java | Total Lines: 123 | Logical Blocks: 25

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the Reservation role in the Restaurant Management architecture.

Lines 1–6 Package & Library Imports

```
1  package com.restaurant.model;
2
3  import com.restaurant.enums.ReservationStatus;
4  import java.time.LocalDate;
5  import java.time.LocalTime;
6
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 6. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 7–7 Class Declaration & Heritage

```
7  public class Reservation {
```

Architecture Role	Defines Reservation as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 7, allowing cross-package invocation.

Lines 8–35 Class Members & Implementation

```
8       private int reservationId;
9       private int customerId;
10      private String customerName;
11      private String customerPhone;
12      private int tableId;
13      private String tableNumber;
14      private LocalDate reservationDate;
15      private LocalTime reservationTime;
16      private int guestCount;
17      private String specialRequest;
18      private ReservationStatus status;
19
20      public Reservation() {
21          this.status = ReservationStatus.PENDING;
22          this.reservationDate = LocalDate.now();
23          this.reservationTime = LocalTime.of(19, 0);
24          this.guestCount = 2;
25      }
26
27      public Reservation(int customerId, int tableId, LocalDate reservationDate, LocalTime reservationTi
me, int guestCount) {
28          this();
29          this.customerId = customerId;
30          this.tableId = tableId;
31          this.reservationDate = reservationDate;
32          this.reservationTime = reservationTime;
33          this.guestCount = guestCount;
34      }
35
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 8–35 (28 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 36–39 Method: getReservationId()

```
36      public int getReservationId() {
37          return reservationId;
38      }
39
```

Component	Method: getReservationId()
Line-by-Line Breakdown	Lines 36–39 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 40–43 Method: setReservationId()

```
40      public void setReservationId(int reservationId) {
41          this.reservationId = reservationId;
42      }
43
```

Component	Method: setReservationId()
Line-by-Line Breakdown	Lines 40–43 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 44–47 Method: getCustomerId()

```
44      public int getCustomerId() {
45          return customerId;
46      }
47
```

Component	Method: getCustomerId()
Line-by-Line Breakdown	Lines 44–47 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 48–51 Method: setCustomerId()

```
48      public void setCustomerId(int customerId) {
49          this.customerId = customerId;
50      }
51
```

Component	Method: setCustomerId()
Line-by-Line Breakdown	Lines 48–51 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 52–55 Method: getCustomerName()

```
52      public String getCustomerName() {
53          return customerName;
54      }
55
```

Component	Method: getCustomerName()
Line-by-Line Breakdown	Lines 52–55 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 56–59 Method: setCustomerName()

```
56      public void setCustomerName(String customerName) {
57          this.customerName = customerName;
58      }
59
```

Component	Method: setCustomerName()
Line-by-Line Breakdown	Lines 56–59 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 60–63 Method: getCustomerPhone()

```
60      public String getCustomerPhone() {
61          return customerPhone;
62      }
63
```

Component	Method: getCustomerPhone()
Line-by-Line Breakdown	Lines 60–63 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 64–67 Method: setCustomerPhone()

```
64      public void setCustomerPhone(String customerPhone) {
65          this.customerPhone = customerPhone;
66      }
67
```

Component	Method: setCustomerPhone()
Line-by-Line Breakdown	Lines 64–67 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 68–71 Method: getTableId()

```
68      public int getTableId() {
69          return tableId;
70      }
71
```

Component	Method: getTableId()
Line-by-Line Breakdown	Lines 68–71 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 72–75 Method: setTableId()

```
72      public void setTableId(int tableId) {
73          this.tableId = tableId;
74      }
75
```

Component	Method: setTableId()
Line-by-Line Breakdown	Lines 72–75 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 76–79 Method: getTableNumber()

```
76      public String getTableNumber() {
77          return tableNumber;
78      }
79
```

Component	Method: getTableNumber()
Line-by-Line Breakdown	Lines 76–79 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 80–83 Method: setTableNumber()

```
80      public void setTableNumber(String tableNumber) {
81          this.tableNumber = tableNumber;
82      }
83
```

Component	Method: setTableNumber()
Line-by-Line Breakdown	Lines 80–83 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 84–87 Method: getReservationDate()

```
84      public LocalDate getReservationDate() {
85          return reservationDate;
86      }
87
```

Component	Method: getReservationDate()
Line-by-Line Breakdown	Lines 84–87 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 88–91 Method: setReservationDate()

```
88      public void setReservationDate(LocalDate reservationDate) {
89          this.reservationDate = reservationDate;
90      }
91
```

Component	Method: setReservationDate()
Line-by-Line Breakdown	Lines 88–91 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 92–95 Method: getReservationTime()

```
92      public LocalTime getReservationTime() {
93          return reservationTime;
94      }
95
```

Component	Method: getReservationTime()
Line-by-Line Breakdown	Lines 92–95 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 96–99 Method: setReservationTime()

```
96      public void setReservationTime(LocalTime reservationTime) {
97          this.reservationTime = reservationTime;
98      }
99
```

Component	Method: setReservationTime()
Line-by-Line Breakdown	Lines 96–99 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 100–103 Method: getGuestCount()

```
100     public int getGuestCount() {
101         return guestCount;
102     }
103
```

Component	Method: getGuestCount()
Line-by-Line Breakdown	Lines 100–103 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 104–107 Method: setGuestCount()

```
104       public void setGuestCount(int guestCount) {
105           this.guestCount = guestCount;
106       }
107
```

Component	Method: setGuestCount()
Line-by-Line Breakdown	Lines 104–107 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 108–111 Method: getSpecialRequest()

```
108       public String getSpecialRequest() {
109           return specialRequest;
110       }
111
```

Component	Method: getSpecialRequest()
Line-by-Line Breakdown	Lines 108–111 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 112–115 Method: setSpecialRequest()

```
112       public void setSpecialRequest(String specialRequest) {
113           this.specialRequest = specialRequest;
114       }
115
```

Component	Method: setSpecialRequest()
Line-by-Line Breakdown	Lines 112–115 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 116–119 Method: getStatus()

```
116       public ReservationStatus getStatus() {
117           return status;
118       }
119
```

Component	Method: getStatus()
Line-by-Line Breakdown	Lines 116–119 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 120–123` Method: setStatus()

```
120       public void setStatus(ReservationStatus status) {
121           this.status = status;
122       }
123   }
```

Component	Method: setStatus()
Line-by-Line Breakdown	Lines 120–123 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

RestaurantTable.java

Path: src\main\java\com\restaurant\model\RestaurantTable.java | Total Lines: 69 | Logical Blocks: 14

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the RestaurantTable role in the Restaurant Management architecture.

Lines 1–4 Package & Library Imports

```
1  package com.restaurant.model;
2
3  import com.restaurant.enums.TableStatus;
4
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 4. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 5–5 Class Declaration & Heritage

```
5  public class RestaurantTable {
```

Architecture Role	Defines RestaurantTable as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 5, allowing cross-package invocation.

Lines 6–24 Class Members & Implementation

```
6       private int tableId;
7       private String tableNumber;
8       private int capacity;
9       private String location;
10      private TableStatus status;
11
12      public RestaurantTable() {
13          this.status = TableStatus.AVAILABLE;
14          this.location = "Main Dining Hall";
15      }
16
17      public RestaurantTable(int tableId, String tableNumber, int capacity, String location, TableStatus
status) {
18          this.tableId = tableId;
19          this.tableNumber = tableNumber;
20          this.capacity = capacity;
21          this.location = location;
22          this.status = status != null ? status : TableStatus.AVAILABLE;
23      }
24
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 6–24 (19 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 25–28 Method: getTableId()

```
25      public int getTableId() {
26          return tableId;
27      }
28
```

Component	Method: getTableId()
Line-by-Line Breakdown	Lines 25–28 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 29–32 Method: setTableId()

```
29      public void setTableId(int tableId) {
30          this.tableId = tableId;
31      }
32
```

Component	Method: setTableId()
Line-by-Line Breakdown	Lines 29–32 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 33–36 Method: getTableNumber()

```
33      public String getTableNumber() {
34          return tableNumber;
35      }
36
```

Component	Method: getTableNumber()
Line-by-Line Breakdown	Lines 33–36 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 37–40 Method: setTableNumber()

```
37      public void setTableNumber(String tableNumber) {
38          this.tableNumber = tableNumber;
39      }
40
```

Component	Method: setTableNumber()
Line-by-Line Breakdown	Lines 37–40 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 41–44 Method: getCapacity()

```
41      public int getCapacity() {
42          return capacity;
43      }
44
```

Component	Method: getCapacity()
Line-by-Line Breakdown	Lines 41–44 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 45–48 Method: setCapacity()

```
45      public void setCapacity(int capacity) {
46          this.capacity = capacity;
47      }
48
```

Component	Method: setCapacity()
Line-by-Line Breakdown	Lines 45–48 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 49–52 Method: getLocation()

```
49      public String getLocation() {
50          return location;
51      }
52
```

Component	Method: getLocation()
Line-by-Line Breakdown	Lines 49–52 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 53–56 Method: setLocation()

```
53      public void setLocation(String location) {
54          this.location = location;
55      }
56
```

Component	Method: setLocation()
Line-by-Line Breakdown	Lines 53–56 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 57–60 Method: getStatus()

```
57      public TableStatus getStatus() {
58          return status;
59      }
60
```

Component	Method: getStatus()
Line-by-Line Breakdown	Lines 57–60 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 61–65 Method: setStatus()

```
61      public void setStatus(TableStatus status) {
62          this.status = status;
63      }
64
65      @Override
```

Component	Method: setStatus()
Line-by-Line Breakdown	Lines 61–65 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 66–69` Method: toString()

```
66      public String toString() {
67          return tableNumber + " (" + capacity + " Seats - " + status.getLabel() + ")";
68      }
69  }
```

Component	Method: toString()
Line-by-Line Breakdown	Lines 66–69 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Supplier.java

Path: src\main\java\com\restaurant\model\Supplier.java | Total Lines: 95 | Logical Blocks: 20

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the Supplier role in the Restaurant Management architecture.

Lines 1–4 Package & Library Imports

```
1  package com.restaurant.model;
2
3  import java.time.LocalDateTime;
4
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 4. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 5–5 Class Declaration & Heritage

```
5  public class Supplier {
```

Architecture Role	Defines Supplier as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 5, allowing cross-package invocation.

Lines 6–26 Class Members & Implementation

```
6       private int supplierId;
7       private String name;
8       private String contactPerson;
9       private String phone;
10      private String email;
11      private String address;
12      private String suppliedCategory;
13      private LocalDateTime createdAt;
14
15      public Supplier() {}
16
17      public Supplier(int supplierId, String name, String contactPerson, String phone, String email, Str
ing address, String suppliedCategory) {
18          this.supplierId = supplierId;
19          this.name = name;
20          this.contactPerson = contactPerson;
21          this.phone = phone;
22          this.email = email;
23          this.address = address;
24          this.suppliedCategory = suppliedCategory;
25      }
26
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 6–26 (21 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 27–30 Method: getSupplierId()

```
27      public int getSupplierId() {
28          return supplierId;
29      }
30
```

Component	Method: getSupplierId()
Line-by-Line Breakdown	Lines 27–30 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 31–34 Method: setSupplierId()

```
31      public void setSupplierId(int supplierId) {
32          this.supplierId = supplierId;
33      }
34
```

Component	Method: setSupplierId()
Line-by-Line Breakdown	Lines 31–34 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 35–38 Method: getName()

```
35      public String getName() {
36          return name;
37      }
38
```

Component	Method: getName()
Line-by-Line Breakdown	Lines 35–38 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 39–42 Method: setName()

```
39      public void setName(String name) {
40          this.name = name;
41      }
42
```

Component	Method: setName()
Line-by-Line Breakdown	Lines 39–42 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 43–46 Method: getContactPerson()

```
43      public String getContactPerson() {
44          return contactPerson;
45      }
46
```

Component	Method: getContactPerson()
Line-by-Line Breakdown	Lines 43–46 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 47–50 Method: setContactPerson()

```
47      public void setContactPerson(String contactPerson) {
48          this.contactPerson = contactPerson;
49      }
50
```

Component	Method: setContactPerson()
Line-by-Line Breakdown	Lines 47–50 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 51–54 Method: getPhone()

```
51      public String getPhone() {
52          return phone;
53      }
54
```

Component	Method: getPhone()
Line-by-Line Breakdown	Lines 51–54 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 55–58 Method: setPhone()

```
55      public void setPhone(String phone) {
56          this.phone = phone;
57      }
58
```

Component	Method: setPhone()
Line-by-Line Breakdown	Lines 55–58 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 59–62 Method: getEmail()

```
59      public String getEmail() {
60          return email;
61      }
62
```

Component	Method: getEmail()
Line-by-Line Breakdown	Lines 59–62 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 63–66 Method: setEmail()

```
63      public void setEmail(String email) {
64          this.email = email;
65      }
66
```

Component	Method: setEmail()
Line-by-Line Breakdown	Lines 63–66 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 67–70 Method: getAddress()

```
67      public String getAddress() {
68          return address;
69      }
70
```

Component	Method: getAddress()
Line-by-Line Breakdown	Lines 67–70 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 71–74 Method: setAddress()

```
71      public void setAddress(String address) {
72          this.address = address;
73      }
74
```

Component	Method: setAddress()
Line-by-Line Breakdown	Lines 71–74 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 75–78 Method: getSuppliedCategory()

```
75      public String getSuppliedCategory() {
76          return suppliedCategory;
77      }
78
```

Component	Method: getSuppliedCategory()
Line-by-Line Breakdown	Lines 75–78 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 79–82 Method: setSuppliedCategory()

```
79      public void setSuppliedCategory(String suppliedCategory) {
80          this.suppliedCategory = suppliedCategory;
81      }
82
```

Component	Method: setSuppliedCategory()
Line-by-Line Breakdown	Lines 79–82 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 83–86 Method: getCreatedAt()

```
83      public LocalDateTime getCreatedAt() {
84          return createdAt;
85      }
86
```

Component	Method: getCreatedAt()
Line-by-Line Breakdown	Lines 83–86 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 87–91 Method: setCreatedAt()

```
87      public void setCreatedAt(LocalDateTime createdAt) {
88          this.createdAt = createdAt;
89      }
90
91      @Override
```

Component	Method: setCreatedAt()
Line-by-Line Breakdown	Lines 87–91 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 92–95 Method: toString()

```
92      public String toString() {
93          return name + " (" + suppliedCategory + ")";
94      }
95  }
```

Component	Method: toString()
Line-by-Line Breakdown	Lines 92–95 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

UPIPayment.java

Path: src\main\java\com\restaurant\model\UPIPayment.java | Total Lines: 61 | Logical Blocks: 7

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the UPIPayment role in the Restaurant Management architecture.

Lines 1–8 Package & Library Imports

```
1  package com.restaurant.model;
2
3  import com.restaurant.enums.PaymentMethod;
4  import com.restaurant.enums.PaymentStatus;
5  import com.restaurant.exception.PaymentFailedException;
6  import com.restaurant.util.CurrencyUtil;
7  import com.restaurant.util.DateTimeUtil;
8
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 8. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 9–9 Class Declaration & Heritage

```
9  public class UPIPayment extends Payment {
```

Architecture Role	Defines UPIPayment as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 9, allowing cross-package invocation.

Lines 10–23 Class Members & Implementation

```
10        private String upiId;
11
12        public UPIPayment() {
13            this.paymentMethod = PaymentMethod.UPI;
14        }
15
16        public UPIPayment(int orderId, String invoiceNumber, double amount, int processedBy, String upiId,
String transactionReference) {
17            super(orderId, invoiceNumber, amount, PaymentMethod.UPI, processedBy);
18            this.upiId = upiId;
19            this.transactionReference = (transactionReference != null && !transactionReference.isEmpty())
20                    ? transactionReference : "UPI-" + System.currentTimeMillis();
21        }
22
23        @Override
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 10–23 (14 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 24–33 Method: processPayment()

```
24        public boolean processPayment() throws PaymentFailedException {
25            if (upiId == null || !upiId.contains("@")) {
26                this.status = PaymentStatus.FAILED;
27                throw new PaymentFailedException("Invalid UPI Virtual Payment Address (VPA): " + upiId);
28            }
29            this.status = PaymentStatus.PAID;
30            return true;
31        }
32
33        @Override
```

Component	Method: processPayment()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 24–33 (10 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 34–53 Method: generateReceipt()

```
34        public String generateReceipt(Order order) {
35            StringBuilder sb = new StringBuilder();
36            sb.append("========================================\n");
37            sb.append("         GRAND FEAST RESTAURANT          \n");
38            sb.append("        104 Gourmet Blvd, Silicon City   \n");
39            sb.append("========================================\n");
40            sb.append(String.format("Invoice No : %s\n", invoiceNumber));
41            sb.append(String.format("Order No   : %s\n", order != null ? order.getOrderNumber() : "ORD-" +
orderId));
42            sb.append(String.format("Date/Time  : %s\n", DateTimeUtil.formatDateTime(paymentDateTime)));
43            sb.append(String.format("Pay Mode   : %s\n", paymentMethod.getDisplayName()));
44            sb.append(String.format("UPI ID     : %s\n", upiId));
45            sb.append(String.format("Txn Ref    : %s\n", transactionReference));
46            sb.append("----------------------------------------\n");
47            sb.append(String.format("Total Paid : %s [SUCCESS]\n", CurrencyUtil.format(amount)));
48            sb.append("----------------------------------------\n");
49            sb.append("       THANK YOU FOR DINING WITH US!     \n");
50            sb.append("========================================\n");
51            return sb.toString();
52        }
53
```

Component	Method: generateReceipt()
Line-by-Line Breakdown	Lines 34–53 (20 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 54–57 Method: getUpiId()

```
54        public String getUpiId() {
55            return upiId;
56        }
57
```

Component	Method: getUpiId()
Line-by-Line Breakdown	Lines 54–57 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 58–61 Method: setUpiId()

```
58        public void setUpiId(String upiId) {
59            this.upiId = upiId;
60        }
61    }
```

Component	Method: setUpiId()
Line-by-Line Breakdown	Lines 58–61 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

User.java

Path: src\main\java\com\restaurant\model\User.java | Total Lines: 134 | Logical Blocks: 25

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the User role in the Restaurant Management architecture.

Lines 1–10 Package & Library Imports

```
1  package com.restaurant.model;
2
3  import com.restaurant.enums.UserRole;
4  import java.time.LocalDateTime;
5  import java.util.List;
6
7  /**
8   * Abstract base class representing a System User.
9   * Demonstrates OOP Encapsulation and Abstraction.
10  */
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 10. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 11–11 Class Declaration & Heritage

```
11  public abstract class User {
```

Architecture Role	Defines User as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 11, allowing cross-package invocation.

Lines 12–42 Class Members & Implementation

```
12        protected int userId;
13        protected String username;
14        protected String passwordHash;
15        protected String salt;
16        protected String fullName;
17        protected String email;
18        protected String phone;
19        protected UserRole role;
20        protected boolean active;
21        protected LocalDateTime createdAt;
22
23        public User() {
24            this.active = true;
25        }
26
27        public User(int userId, String username, String fullName, String email, String phone, UserRole rol
e) {
28            this.userId = userId;
29            this.username = username;
30            this.fullName = fullName;
31            this.email = email;
32            this.phone = phone;
33            this.role = role;
34            this.active = true;
35        }
36
37        /**
38         * Abstract method demonstrating polymorphism across user roles.
39         * @return list of permitted action names for this role.
40         */
41        public abstract List<String> getPermissions();
42
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 12–42 (31 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 43–49 Method: hasPermission()

```
43        public boolean hasPermission(String permission) {
44            if (permission == null) return false;
45            List<String> perms = getPermissions();
46            return perms.contains(permission) || perms.contains("ALL");
47        }
48
49        // Encapsulation: Getters and Setters
```

Component	Method: hasPermission()
Line-by-Line Breakdown	Lines 43–49 (7 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 50–53 Method: getUserId()

```
50      public int getUserId() {
51          return userId;
52      }
53
```

Component	Method: getUserId()
Line-by-Line Breakdown	Lines 50–53 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 54–57 Method: setUserId()

```
54      public void setUserId(int userId) {
55          this.userId = userId;
56      }
57
```

Component	Method: setUserId()
Line-by-Line Breakdown	Lines 54–57 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 58–61 Method: getUsername()

```
58      public String getUsername() {
59          return username;
60      }
61
```

Component	Method: getUsername()
Line-by-Line Breakdown	Lines 58–61 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 62–65 Method: setUsername()

```
62      public void setUsername(String username) {
63          this.username = username;
64      }
65
```

Component	Method: setUsername()
Line-by-Line Breakdown	Lines 62–65 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 66–69 Method: getPasswordHash()

```
66      public String getPasswordHash() {
67          return passwordHash;
68      }
69
```

Component	Method: getPasswordHash()
Line-by-Line Breakdown	Lines 66–69 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 70–73 Method: setPasswordHash()

```
70      public void setPasswordHash(String passwordHash) {
71          this.passwordHash = passwordHash;
72      }
73
```

Component	Method: setPasswordHash()
Line-by-Line Breakdown	Lines 70–73 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 74–77 Method: getSalt()

```
74      public String getSalt() {
75          return salt;
76      }
77
```

Component	Method: getSalt()
Line-by-Line Breakdown	Lines 74–77 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 78–81 Method: setSalt()

```
78      public void setSalt(String salt) {
79          this.salt = salt;
80      }
81
```

Component	Method: setSalt()
Line-by-Line Breakdown	Lines 78–81 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 82–85 Method: getFullName()

```
82      public String getFullName() {
83          return fullName;
84      }
85
```

Component	Method: getFullName()
Line-by-Line Breakdown	Lines 82–85 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 86–89 Method: setFullName()

```
86      public void setFullName(String fullName) {
87          this.fullName = fullName;
88      }
89
```

Component	Method: setFullName()
Line-by-Line Breakdown	Lines 86–89 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 90–93 Method: getEmail()

```
90      public String getEmail() {
91          return email;
92      }
93
```

Component	Method: getEmail()
Line-by-Line Breakdown	Lines 90–93 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 94–97 Method: setEmail()

```
94      public void setEmail(String email) {
95          this.email = email;
96      }
97
```

Component	Method: setEmail()
Line-by-Line Breakdown	Lines 94–97 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 98–101 Method: getPhone()

```
 98      public String getPhone() {
 99          return phone;
100      }
101
```

Component	Method: getPhone()
Line-by-Line Breakdown	Lines 98–101 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 102–105 Method: setPhone()

```
102      public void setPhone(String phone) {
103          this.phone = phone;
104      }
105
```

Component	Method: setPhone()
Line-by-Line Breakdown	Lines 102–105 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 106–109 Method: getRole()

```
106      public UserRole getRole() {
107          return role;
108      }
109
```

Component	Method: getRole()
Line-by-Line Breakdown	Lines 106–109 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 110–113 Method: setRole()

```
110      public void setRole(UserRole role) {
111          this.role = role;
112      }
113
```

Component	Method: setRole()
Line-by-Line Breakdown	Lines 110–113 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 114–117 Method: isActive()

```
114       public boolean isActive() {
115           return active;
116       }
117
```

Component	Method: isActive()
Line-by-Line Breakdown	Lines 114–117 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 118–121 Method: setActive()

```
118       public void setActive(boolean active) {
119           this.active = active;
120       }
121
```

Component	Method: setActive()
Line-by-Line Breakdown	Lines 118–121 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 122–125 Method: getCreatedAt()

```
122       public LocalDateTime getCreatedAt() {
123           return createdAt;
124       }
125
```

Component	Method: getCreatedAt()
Line-by-Line Breakdown	Lines 122–125 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 126–130 Method: setCreatedAt()

```
126       public void setCreatedAt(LocalDateTime createdAt) {
127           this.createdAt = createdAt;
128       }
129
130       @Override
```

Component	Method: setCreatedAt()
Line-by-Line Breakdown	Lines 126–130 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 131–134` Method: toString()

```
131        public String toString() {
132            return fullName + " (" + role + ")";
133        }
134    }
```

Component	Method: toString()
Line-by-Line Breakdown	Lines 131–134 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Waiter.java

Path: src\main\java\com\restaurant\model\Waiter.java | Total Lines: 20 | Logical Blocks: 4

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the Waiter role in the Restaurant Management architecture.

Lines 1–5 Package & Library Imports

```
1  package com.restaurant.model;
2
3  import com.restaurant.enums.UserRole;
4  import java.util.List;
5
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 5. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 6–6 Class Declaration & Heritage

```
6  public class Waiter extends User {
```

Architecture Role	Defines Waiter as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 6, allowing cross-package invocation.

Lines 7–16 Class Members & Implementation

```
7
8      public Waiter() {
9          this.role = UserRole.WAITER;
10     }
11
12     public Waiter(int userId, String username, String fullName, String email, String phone) {
13         super(userId, username, fullName, email, phone, UserRole.WAITER);
14     }
15
16     @Override
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 7–16 (10 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 17–20` Method: getPermissions()

```
17        public List<String> getPermissions() {
18            return List.of("VIEW_TABLES", "CREATE_ORDER", "MODIFY_ORDER", "VIEW_MENU", "SEND_TO_KITCHEN",
"VIEW_ORDER_STATUS");
19        }
20    }
```

Component	Method: getPermissions()
Line-by-Line Breakdown	Lines 17–20 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

OrderStatus.java

Path: src\main\java\com\restaurant\enums\OrderStatus.java | Total Lines: 27 | Logical Blocks: 5

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the OrderStatus role in the Restaurant Management architecture.

Lines 1–2 Package & Library Imports

```
1   package com.restaurant.enums;
2
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 2. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 3–3 Class Declaration & Heritage

```
3   public enum OrderStatus {
```

Architecture Role	Defines OrderStatus as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 3, allowing cross-package invocation.

Lines 4–19 Class Members & Implementation

```
4       CREATED("Created", 1),
5       CONFIRMED("Confirmed", 2),
6       PREPARING("Preparing in Kitchen", 3),
7       READY("Ready for Service", 4),
8       SERVED("Served to Table", 5),
9       COMPLETED("Completed & Paid", 6),
10      CANCELLED("Cancelled", 0);
11
12      private final String description;
13      private final int sequence;
14
15      OrderStatus(String description, int sequence) {
16          this.description = description;
17          this.sequence = sequence;
18      }
19
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 4–19 (16 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 20–23` Method: getDescription()

```
20      public String getDescription() {
21          return description;
22      }
23
```

Component	Method: getDescription()
Line-by-Line Breakdown	Lines 20–23 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 24–27` Method: getSequence()

```
24      public int getSequence() {
25          return sequence;
26      }
27  }
```

Component	Method: getSequence()
Line-by-Line Breakdown	Lines 24–27 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

PaymentMethod.java

Path: src\main\java\com\restaurant\enums\PaymentMethod.java | Total Lines: 18 | Logical Blocks: 4

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the PaymentMethod role in the Restaurant Management architecture.

Lines 1–2 Package & Library Imports

```
1  package com.restaurant.enums;
2
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 2. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 3–3 Class Declaration & Heritage

```
3  public enum PaymentMethod {
```

Architecture Role	Defines PaymentMethod as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 3, allowing cross-package invocation.

Lines 4–14 Class Members & Implementation

```
4      CASH("Cash Settlement"),
5      UPI("Unified Payments Interface (UPI)"),
6      CREDIT_CARD("Credit Card"),
7      DEBIT_CARD("Debit Card");
8
9      private final String displayName;
10
11     PaymentMethod(String displayName) {
12         this.displayName = displayName;
13     }
14
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 4–14 (11 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 15–18` Method: getDisplayName()

```
15      public String getDisplayName() {
16          return displayName;
17      }
18  }
```

Component	Method: getDisplayName()
Line-by-Line Breakdown	Lines 15–18 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

PaymentStatus.java

Path: src\main\java\com\restaurant\enums\PaymentStatus.java | Total Lines: 18 | Logical Blocks: 4

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the PaymentStatus role in the Restaurant Management architecture.

Lines 1–2 Package & Library Imports

```
1   package com.restaurant.enums;
2
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 2. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 3–3 Class Declaration & Heritage

```
3   public enum PaymentStatus {
```

Architecture Role	Defines PaymentStatus as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 3, allowing cross-package invocation.

Lines 4–14 Class Members & Implementation

```
4       PENDING("Payment Pending"),
5       PAID("Paid & Settled"),
6       FAILED("Payment Failed"),
7       REFUNDED("Refunded");
8
9       private final String description;
10
11      PaymentStatus(String description) {
12          this.description = description;
13      }
14
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 4–14 (11 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 15–18` Method: getDescription()

```
15        public String getDescription() {
16            return description;
17        }
18    }
```

Component	Method: getDescription()
Line-by-Line Breakdown	Lines 15–18 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

ReservationStatus.java

Path: src\main\java\com\restaurant\enums\ReservationStatus.java | Total Lines: 18 | Logical Blocks: 4

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the ReservationStatus role in the Restaurant Management architecture.

Lines 1–2 Package & Library Imports

```
1   package com.restaurant.enums;
2
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 2. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 3–3 Class Declaration & Heritage

```
3   public enum ReservationStatus {
```

Architecture Role	Defines ReservationStatus as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 3, allowing cross-package invocation.

Lines 4–14 Class Members & Implementation

```
4       PENDING("Pending Confirmation"),
5       CONFIRMED("Confirmed"),
6       CANCELLED("Cancelled"),
7       COMPLETED("Completed");
8
9       private final String label;
10
11      ReservationStatus(String label) {
12          this.label = label;
13      }
14
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 4–14 (11 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 15–18` Method: getLabel()

```
15        public String getLabel() {
16            return label;
17        }
18    }
```

Component	Method: getLabel()
Line-by-Line Breakdown	Lines 15–18 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

TableStatus.java

Path: src\main\java\com\restaurant\enums\TableStatus.java | Total Lines: 24 | Logical Blocks: 5

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the TableStatus role in the Restaurant Management architecture.

Lines 1–2 Package & Library Imports

```
1   package com.restaurant.enums;
2
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 2. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 3–3 Class Declaration & Heritage

```
3   public enum TableStatus {
```

Architecture Role	Defines TableStatus as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 3, allowing cross-package invocation.

Lines 4–16 Class Members & Implementation

```
4       AVAILABLE("Available", "#2ECC71"),
5       OCCUPIED("Occupied", "#E74C3C"),
6       RESERVED("Reserved", "#F39C12"),
7       CLEANING("Cleaning", "#3498DB");
8
9       private final String label;
10      private final String colorHex;
11
12      TableStatus(String label, String colorHex) {
13          this.label = label;
14          this.colorHex = colorHex;
15      }
16
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 4–16 (13 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 17–20` Method: getLabel()

```
17      public String getLabel() {
18          return label;
19      }
20
```

Component	Method: getLabel()
Line-by-Line Breakdown	Lines 17–20 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 21–24` Method: getColorHex()

```
21      public String getColorHex() {
22          return colorHex;
23      }
24  }
```

Component	Method: getColorHex()
Line-by-Line Breakdown	Lines 21–24 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

UserRole.java

Path: src\main\java\com\restaurant\enums\UserRole.java | Total Lines: 25 | Logical Blocks: 5

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the UserRole role in the Restaurant Management architecture.

Lines 1–2 Package & Library Imports

```
1  package com.restaurant.enums;
2
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 2. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 3–3 Class Declaration & Heritage

```
3  public enum UserRole {
```

Architecture Role	Defines UserRole as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 3, allowing cross-package invocation.

Lines 4–17 Class Members & Implementation

```
4      ADMIN("Administrator", "Full system access"),
5      MANAGER("Manager", "Management of menu, tables, inventory and analytics"),
6      WAITER("Waiter / Server", "Order creation, table seating and kitchen dispatch"),
7      CASHIER("Cashier", "Billing calculation, receipts and payment processing"),
8      KITCHEN_STAFF("Kitchen Staff", "Live kitchen display and order status updates");
9
10     private final String displayName;
11     private final String description;
12
13     UserRole(String displayName, String description) {
14         this.displayName = displayName;
15         this.description = description;
16     }
17
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 4–17 (14 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 18–21 Method: getDisplayName()

```
18        public String getDisplayName() {
19            return displayName;
20        }
21
```

Component	Method: getDisplayName()
Line-by-Line Breakdown	Lines 18–21 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 22–25 Method: getDescription()

```
22        public String getDescription() {
23            return description;
24        }
25    }
```

Component	Method: getDescription()
Line-by-Line Breakdown	Lines 22–25 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.